

Blockchain-Assisted Fine-Grained Deduplication and Integrity Auditing for Outsourced Large-Scale Data in Cloud Storage

Bingyun Liu^{ID}, Xiaojun Zhang^{ID}, Xingchun Yang^{ID}, Yuan Zhang^{ID}, *Member, IEEE*,
Jingting Xue^{ID}, and Rang Zhou^{ID}

Abstract—Cloud computing has emerged as a promising mode for storing vast quantities of big data, which is vulnerable to potential security threats, making it urgent to ensure data confidentiality and integrity auditing. In addition, as a large number of duplicate data files exist in cloud storage, data deduplication is significant to improve storage efficiency. In this article, we propose a blockchain-assisted fine-grained deduplication and integrity auditing scheme for outsourced large-scale data in cloud storage, achieving internal deduplication and cross-user external deduplication for ciphertexts and authentication tags. By constructing sparse summation ciphertext tree, the scheme implements Proofs of Ownership protocol through vector commitment, and guarantees the retrieval of distributed deduplication data blocks by designing the reconstruction matrix and bloom filter. The scheme exploits blockchain and smart contracts to ensure transparent integrity auditing without a third-party auditor (TPA), thereby avoiding malicious auditing biases. Security analysis and performance evaluation demonstrate the feasibility of the scheme for deploying in cloud storage systems.

Index Terms—Blockchain, cloud computing, fine-grained deduplication, integrity auditing.

Received 27 December 2024; revised 10 February 2025; accepted 1 March 2025. Date of publication 6 March 2025; date of current version 9 June 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106503; in part by the National Nature Science Foundation of China under Grant 61902327 and Grant 62102048; in part by the Open Project of National Engineering Research Center of Oil & Gas Exploration Computer Software under Grant DFWT-ZYRJ2024-JS-81; in part by the Natural Science Foundation of Sichuan Province under Grant 2025ZNSFSC0495 and Grant 2023NSFSC1399; and in part by Intelligent Policing Key Laboratory of Sichuan Province under Grant ZNJW2025KFMS006. (Corresponding authors: Xingchun Yang; Xiaojun Zhang.)

Bingyun Liu, Xiaojun Zhang, and Jingting Xue are with the School of Computer Science and Software Engineering, Engineering Research Center for Intelligent Oil and Gas Exploration, Southwest Petroleum University, Chengdu 610500, China (e-mail: lby_swpu@163.com; zhangxjdzkd2012@163.com; jtxue@swpu.edu.cn).

Xingchun Yang is with the Intelligent Policing Key Laboratory of Sichuan Province, Sichuan Police College, Luzhou 646000, China (e-mail: xingchun-yang@scpolicecc.edu.cn).

Yuan Zhang is with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: ZY_LoYe@126.com).

Rang Zhou is with the school of Computer Science and Cyber Security, Chengdu University of Technology, Chengdu 610059, China (e-mail: zr02018@hotmail.com).

Digital Object Identifier 10.1109/IJOT.2025.3548681

I. INTRODUCTION

OWING to significant technological advancements, various industries are producing vast quantities of big data that have surpassed the capabilities of traditional storage systems [1]. The emergence of cloud computing offers a beneficial solution for enterprises and users in managing big data, effectively easing the burden on local storage infrastructure [2], [3]. Users are willing to outsource their data to cloud servers, but once data are outsourced, they become physically uncontrollable. On the one hand, cloud service providers (CSPs) encounter threats, such as hardware and software failures, hacking attacks, and other malicious activities, which may lead to the loss of stored data and data leakage. On the other hand, to optimize storage space for greater interest, CSPs may deliberately delete data which are infrequently used, thereby compromising data integrity. As such, users will be more concerned about data confidentiality and integrity [4]. Additionally, as the user base expands, the International Data Center reports that cloud storage data will surpass 175 ZB by 2025, with over 50% of data being duplicate [5], where it is a huge waste of storage space. Despite the vast storage capacity offered by CSPs, achieving maximum storage efficiency by secure and reliable data deduplication holds significantly practical importance, to save storage space, CSPs may perform malicious deduplication while directly deleting some unique data blocks, resulting in unavailability of outsourced data by end users.

To ensure data confidentiality, the most direct approach is to encrypt outsourced data. To prevent CSPs from falsifying outsourced data to deceive users, based on the provable data possession (PDP) in [6], remote data integrity auditing schemes have been proposed [7], [8], [9], [10], [11], [12], [13]. But, the private auditing schemes in [7], [8], and [9] increase the user-side costs, the public auditing schemes in [10], [11], [12], and [13] are assisted by introducing a third-party auditor (TPA) to complete the integrity auditing, which will increase the economic costs. And most of them are based on a strong assumption that TPA is honest and reliable. However, TPA is inquisitive and it also generates malicious auditing behaviors. Thus, blockchain could be integrated into integrity auditing to avoid such malicious auditing biases.

For CSPs, many deduplication mechanisms [14], [15], [16] have been designed to improve storage efficiency. Although some deduplication mechanisms have been exploited by CSPs,

such as Google cloud [17], only a single interface has been implemented, without considering the integrity checking for outsourced data. More importantly, the previous integrity auditing schemes cannot be directly combined with the CSP that implements the data deduplication. As such, it is significant for users to execute a mechanism supporting data integrity auditing and deduplication, such that both parties can enjoy or provide secure storage services. Recently, there exist some schemes [8], [18], [19], [20], [21], [22], [23] combining integrity auditing and deduplication, which mainly focus on the implementation of authentication tags and ciphertext deduplication in file-level. However, there exist areas that need to be optimized in integrity auditing or data deduplication. For example, users usually do not want to disclose ownership privacy (OP) in deduplication. However, in [21], [22], and [23], there exist adversaries which can obtain OP from public information. In [20], [22], and [23], they consider duplication fake attacks (DFAs) by the initial uploader and low-entropy data auditing reliability. Because the initial uploader may upload authentication tags that do not match the ciphertext such that subsequent uploaders are deceived, and it is easy for an adversary to obtain the signature key of low-entropy data through brute-force attacks (BFAs). In [19], it has high computational costs on each user side and requires a user to stay online in real time during the integrity auditing process. In resource-constrained devices, it tends to cause excessive computational burden, which is infeasible in practice. Moreover, as these schemes achieve deduplication and integrity auditing at the file level, to improve the storage efficiency of CSPs, fine-grained deduplication should be a better candidate approach.

To cope with these issues, we propose a blockchain-assisted fine-grained deduplication and integrity auditing (BA-FGDIA) scheme for large-scale data in cloud storage, termed as BA-FGDIA. The contributions of this article are elaborated as follows.

- 1) We devise the fine-grained data deduplication and integrity auditing mechanism, which achieves both internal deduplication and cross-user external deduplication. In particular, to improve fine-grained deduplication motivation and avoid malicious deduplication, a user can calculate a deduplication credential by bitstream of detection result. Thus, a new charge strategy of outsourced file is jointly determined by outsourced data and the deduplication credential, the number of deduplication data blocks is higher, the lower outsourced charge for users.
- 2) The sparse summation verkle tree (SSVT) is exploited to design a ciphertext tree, enabling the utilization of vector commitment for generating fixed size of Proofs of Ownership (PoW) for distributed backup blocks. Furthermore, we design the ciphertext reconstruction matrix and bloom filter to achieve the location of backup ciphertexts, the reconstruction and recovery of duplication data blocks in file.
- 3) We integrate blockchain systems and proxy-oriented mechanism into the transparent integrity auditing of single file and batch files, in which we rely on the

characteristics of blockchain to deploy smart contracts and proxy resignatures to undertake different integrity auditing tasks, thereby eliminating the need for TPA and effectively avoiding the deceptive behaviors.

- 4) The security analysis demonstrates BA-FGDIA simultaneously achieves data confidentiality, OP protection, and reliability of integrity auditing. The performance comparison with existing schemes show BA-FGDIA has lower computational costs on each user side and lightweight integrity auditing costs via blockchain systems.

II. RELATED WORK

Since outsourced data is out of control for users, integrity auditing provides users with an approach to check outsourced data integrity. Juels and Kaliski [24] proposed proofs of retrievability (POR) mechanism to check data integrity. Ateniese et al. [6] introduced the PDP mechanism that achieves the integrity auditing without retrieving the remote data. Shacham and Waters [25] improved the PDP mechanism, more integrity auditing schemes have been proposed [7], [8], [9], requiring to check data integrity by themselves, which leads to substantial computation burden on users. To address this issue, the public integrity auditing schemes deploying the TPA have been proposed [10], [11], [12], [13]. However, these schemes depend on a strong assumption that TPA is fully trusted. Actually, TPA may conspire with CSPs to provide a biased auditing result. In [26], they primarily place proof information and auditing logs as transactions on the public blockchain, after TPA checks data integrity, a user can verify whether there exist malicious behaviors. Zhang et al. [27] proposed a proxy-oriented data integrity auditing scheme in intelligent transportation systems.

In addition, encrypted data deduplication not only improves cloud storage efficiency but also ensures data confidentiality. The traditional encryption method cannot be used in data deduplication, because the ciphertext of the same data owned by different users is usually generated by different keys, making it impossible to detect duplication in ciphertexts. Convergent encryption (CE) is proposed by Douceur et al. [14], where they advocate to exploit the hash value of plaintexts as encryption keys. Later, Bellare et al. [15] formalized Douceur et al.'s work and first proposed message locking encryption technology to implement data deduplication. Based on the work, many deduplication mechanisms [16], [28], [29] have emerged. However, they are vulnerable to offline BFAs. Thus, [30] proposed the deduplication mechanism in which they use the key server to generate the encryption keys to resist BFA, yet there exists single-point failure. To address single-point failure, [18] and [31] introduced the deduplication mechanisms supporting password-security encryption key, where they use the key server group and periodically change group members to generate keys. Additionally, the deduplication strategy is divided into client-side deduplication and server-side deduplication. Although the former can reduce a user's computation and communication costs, it is vulnerable to BFA and requires to achieve PoW. As such, Halevi et al. [32] proposed a PoW

protocol to prevent adversaries deceiving CSP by snooping file tag. However, these mechanisms only consider data deduplication, but in practical applications, users also concern whether data integrity and availability are guaranteed after deduplication.

Yuan and Yu [33] proposed the mechanism achieving integrity auditing and deduplication, in which they perform the deduplication on plaintexts such that the inability to guarantee data confidentiality and require all users to upload authentication tags. Xu et al. [23] proposed the blockchain-assisted data integrity auditing and deduplication plaintexts and authentication tags, yet does not consider the data confidentiality. Yuan et al. [22] introduced blockchain-based public auditing and secure deduplication mechanism. Although they guarantee the data confidentiality, they do not consider the leakage of users' OP in the deduplication. Zhang et al. [34] exploited blockchain to implement a double copy storage mode to achieve public auditing for deduplication, and also achieve OP protection. However, it has the risk of data leakage, and cannot resist DFA attack of the initial uploader. Song et al. [20] proposed transparent deduplication and integrity auditing scheme, where they achieve the OP protection and verify the data integrity without TPA, yet they directly use the hash of file contents as the signing key, making it impossible to protect the key security of low-entropy file and ensure the reliability of auditing results. Miao et al. [35] proposed an ID-based deduplication and auditing scheme in which they rely on blockchain to achieve integrity auditing without TPA and resist DFA attack of the initial uploader. Subsequently, Gao et al. [21] devised the low-entropy data deduplication and integrity auditing scheme. They generate authentication tags by using random keys chosen by the initial uploader, however, this scheme cannot guarantee the auditing results of subsequent uploaders and monitor the malicious behaviors of TPA. Recently, Zheng et al. [36] proposed a secure deduplication and integrity auditing scheme supporting dynamic operations, in which a lightweight authenticator structure is designed to reduce the costs of integrity auditing. However, it does not take into account the auditing bias of TPA and the security problems of key generation, because the key of low-entropy file is easy to be cracked by the adversary through brute force. Song et al. [19] proposed a blockchain-based deduplication and integrity auditing scheme, which solves the key security problem of low entropy data by introducing random factors.

III. PRELIMINARIES

A. Bilinear Pairing and Hardness Assumptions

Let G_1 and G_T be multiplicative cycle groups of the large prime order q . Define a bilinear pairing map $e: G_1 \times G_1 \rightarrow G_T$, satisfying the following properties.

- 1) *Bilinearity*: Given two elements $a, b \in Z_q^*$ and $g_1, g_2 \in G_1$, $e(g_1^a, g_2^b) = e(g_1, g_2)^{ab}$.
- 2) *Nondegenerability*: There exist two elements $g_1, g_2 \in G_1$, such that $e(g_1, g_2) \neq 1 \in G_T$.
- 3) *Computability*: For any $g_1, g_2 \in G_1$, it is computationally feasible to compute $e(g_1, g_2)$ in polynomial time.

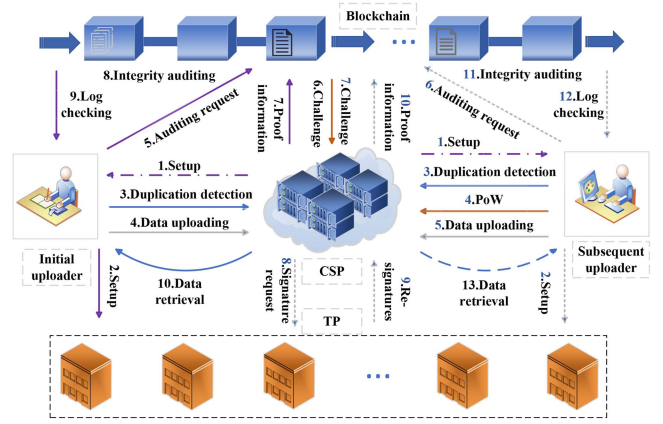


Fig. 1. System model of BA-FGDIA.

Discrete Logarithm (DL) Problem: Given a q -order multiplicative cyclic group G_1 with a generator g . The goal of DL problem is to compute $a \in Z_q$ based on random instance (g, g^a) .

Computational Diffie-Hellman (CDH) Problem: Given a q -order multiplicative cyclic group G_1 with a generator g . Let $a, b \in Z_q^*$, the goal of CDH problem is to compute g^{ab} based on (g, g^a, g^b) .

Essentially, it is computationally infeasible to solve the hardness assumptions of DL and CDH in polynomial time.

B. Sparse Summation Verkle Tree

SSVT [37] is a novel authentication data structure with the size as $\mathcal{T}_{N,d}$, where the arity $N \geq 2$, d is the depth. Each inner node stores two vector commitments V and W .

- 1) Commitment V is computed with a $N+1$ length vector v , in which $v[i]$ is the leaf node value of i -th child and $v[N] = \sum_{i=0}^{N-1} v[i]$.
- 2) W is a vector commitment corresponded to w length vector, in which $w[i]$ is the hash value of i -th child vector commitment if the child node is inner node. Otherwise, $w[i] = \phi$.

C. Homomorphic Hash Function

We introduce the homomorphic hash function $H(\cdot)$ [27], satisfying the following.

- 1) *Homomorphic Property*: For any m_1 and m_2 , choose two scalars α, β , H satisfies

$$H(\alpha m_1 + \beta m_2) = H(m_1)^\alpha \cdot H(m_2)^\beta.$$

- 2) *Collision Resistance*: Given m_3 and $\alpha m_1 + \beta m_2$, where $m_3 \neq \alpha m_1 + \beta m_2$, $H(m_3) = H(m_1)^\alpha \cdot H(m_2)^\beta$.

D. System Model

The system model of BA-FGDIA is depicted in Fig. 1, including four types of entities: CSP, users, trusted proxy (TP), and blockchain.

- 1) *CSP*: It executes data duplication, stores outsourced data, generates the proof information about the number of deduplicated data blocks and integrity auditing

proof. In addition, CSP puts the proof information on blockchain to trigger smart contracts in integrity auditing.

- 2) *Users*: Each user computes the deduplication tags and outsources encrypted data to CSP, triggers smart contracts to execute integrity auditing. Each user could check the log information on blockchain and retrieve the outsourced data. In particular, each user will be divided into initial uploader outsourcing no duplicate new file and subsequent upload outsourcing a data file, in which some data blocks have been stored in the cloud server.
- 3) *TP*: Each TP verifies the resignature token and computes the authentication tags corresponded to the challenge data blocks that have been deduplicated.
- 4) *Blockchain*: The blockchain provides random sources that rely on the unpredictability of blocks generation, stores integrity auditing auxiliary information and auditing log information.

The security threats of BA-FGDIA come from the aspects of users, cloud server and proxies, as described as follows.

- 1) *Semi-Honest Users*: For legitimate users, they can access public information on the blockchain. A user may snoop the OP of other users from the auditing logs.
- 2) *Semi-Honest CSP*: CSP may snoop the contents of outsourced data. The outsourced data may be lost due to hardware failures, CSP may forge encrypted data and authentication tags to pass integrity auditing to deceive users. Besides, in BA-FGDIA, we require that CSP would not collude with the proxies or any users for the reason of the commercial reputation as claimed in [19].
- 3) *Semi-Honest Proxies*: Upon receiving the resignature generation request, the curious proxy may snoop the OP of a user from the signature request.

E. Design Goals

The BA-FGDIA scheme for outsourced large-scale data should achieve the following design goals.

- 1) *Functionality*: BA-FGDIA has two main functions: *fine-grained deduplication* and *integrity auditing*. In particular, CSP deduplicates both ciphertexts and authentication tags of data blocks for a data file, achieves one-time integrity auditing request to simultaneously verify the number of deduplication data blocks and outsourced data integrity auditing. In addition, CSP could recover the complete file after fine-grained deduplication.
- 2) *Security*: BA-FGDIA guarantees data confidentiality such that CSP cannot exploit its knowledge to obtain file contents and encryption keys. BA-FGDIA guarantees OP protection such that an adversary cannot disclose the OP of a user through public information, and it cannot know which the files of a user have duplicated data blocks or which users own identical duplicated data blocks. BA-FGDIA guarantees that forged signatures or ciphertext blocks will not pass integrity auditing.
- 3) *High Performance*: BA-FGDIA is committed to reducing the computational costs and storage costs on each

user side when files are outsourced. BA-FGDIA should achieve more lightweight computation advantages in integrity auditing for signal file and batch files.

IV. CONSTRUCTION OF BA-FGDIA

In this section, we present the BA-FGDIA scheme for outsourced large-scale data. BA-FGDIA consists of the following seven polynomial time algorithms: 1) *System Initialization*; 2) *Duplication Detection*; 3) *Initial Uploading*; 4) *Subsequent Uploading*; 5) *Integrity Auditing*; 6) *Log Checking*; and 7) *Data Retrieval*.

A. System Initialization

- 1) CSP initializes the system public parameters as follows.

- a) Randomly choose three security parameters $\kappa_0, \kappa_1, \kappa_2$, generate a large prime $p = 3 \pmod{4}$ based on κ_0 , determine two multiplicative cyclic groups G_1 and G_2 of order p , set a bilinear pairing map $e : G_1 \times G_1 \rightarrow G_2$, select g as the generator of G_1 and randomly select $W \in G_1$. Randomly select $sk_c = \varsigma \in Z_p^*$ as the secret key and compute $pk_c = g^{\varsigma^{-1}}$ as corresponding public key.
- b) Define $\Gamma(\cdot) : \mathcal{H}^* \rightarrow Z_p^*$ as an invertible mapping function, such that $\Gamma^{-1}(\cdot) \in \mathcal{H}^*$, set an invertible iterative function $\tau = \rho \circ \gamma$, where ρ and γ are two permutations as described in [38].
- c) Define collision-resistant hash functions: $h_1(\cdot) : \{0, 1\}^* \rightarrow \mathcal{H}^{\kappa_1}$, $h_2(\cdot) : G_1 \times G_1 \times Z_p^* \rightarrow \{0, 1\}^{\kappa_2}$, $h_3(\cdot) : \{0, 1\}^* \rightarrow Z_p^*$, $h_4(\cdot) : G_1 \times \{0, 1\}^* \rightarrow \{0, 1, \dots, \mathbb{N} - 1\}^d$ and $h_5(\cdot) : \{0, 1\}^* \rightarrow \{0, 1\}^l$. Determine a short hash function $SH(\cdot) : Z_p^* \times \{0, 1\}^* \rightarrow \{1, \dots, \lfloor (N/2) \rfloor\}$, a homomorphic hash function $H(\cdot) : Z_p^* \rightarrow G_1$, and a count bloom filter $CBF_{(w, l)}$, where w is the size of the array and l is the number of hash functions.
- d) Determine a pseudorandom permutation function $\pi_0(\cdot) : \{1, \dots, n\} \times Z_p^* \rightarrow \{1, \dots, n\}$, two pseudorandom functions $f_0(\cdot) : \{0, 1\}^* \times \{0, \dots, N - 1\} \rightarrow Z_p^*$, $f_1(\cdot) : \{1, \dots, n\} \times Z_p^* \rightarrow Z_p^*$, and a symmetric encryption algorithm Enc .
- e) Randomly select N prime numbers $q_0 < q_1 < \dots < q_{N-1} < p$, satisfying $\gcd(q_i, q_j) = 1$ and $\gcd(q_i, p) = 1, i, j \in [0, N - 1]$, set N proxies $P_1, P_2, \dots, P_N$, compute $\Psi = q_0 \cdot q_1 \cdot \dots \cdot q_{N-1}$.

CSP publishes the system public parameters set $pp = \{e, G_1, G_2, p, pk_c, g, W, \Gamma, H, h_1, h_2, h_3, h_4, h_5, \pi_0, f_0, f_1, SH, Enc, CBF\}$.

- 2) A user generates the secret parameters, distributes sub-secrets to proxies as follows.
 - a) The user enjoying the cloud storage service is assigned an identity $id \in \{0, 1\}^*$. CSP randomly selects $r \in Z_p^*$, computes $E = H(\Psi)^{\Psi^{-1}sk_c} \cdot g^{rh_3(\Psi || \text{Time}_0)}$ and $I = pk_c^r$, where Time_0 is the timestamp. CSP sends the auxiliary information $\Delta = \{\{q_i\}_{i \in [0, N-1]}, \Psi, I, E, \text{Time}_0\}$ to id .
 - b) Upon receiving Δ , id obtains the current timestamp Time_1 to judge $\text{Time}_1 - \text{Time}_0 \leq DT$, then verifies

Algorithm 1 Generation of $D_{Nodup}^{(id,F)}$ and $D_{Dup}^{(id,F)}$

Require: $DB^{(id,F)}$.**Ensure:** Nonduplication set $D_{Nodup}^{(id,F)}$ and the duplication set $D_{Dup}^{(id,F)}$.

```

For  $i \in [1, |DB^{(id,F)}|]$  do
     $tag_i^{(id,F)} = e(tag_i^{(id,F)}, pk_1^{(id)} \cdot pk_1^{(id)^{-1}})$ ;
     $DB^{(id,F)}[i].replace(tag_i^{(id,F)})$ ;
 $D_{Dup}^{(id,F)} = NULL$ ;
 $D_{Nodup}^{(id,F)} = D_{Nodup}^{(id,F)}.add(tag_1^{(id,F)})$ ;
% CSP could judge the internal deduplication.
For  $i \in [2, |DB^{(id,F)}|]$  do
    For  $j \in [1, |DB_{Nodup}^{(id,F)}|]$  do
        if  $tag_i^{(id,F)} \neq tag_j^{(id,F)}$  then;
             $DB_{Nodup}^{(id,F)}.add(tag_i^{(id,F)})$ ;
% CSP could judge the external deduplicate tags in  $F$  by comparing
% with deduplication tags uploaded by previous users.
For  $i \in [1, |DB_{Nodup}^{(id,F)}|]$  do
    For  $j \in [1, |DB|]$  do
        if  $tag_i^{(id,F)} = tag_j^{(id^*,F^*)}$  then;
             $DB_{Nodup}^{(id,F)}.delete(tag_i^{(id,F)})$ ;
             $DB_{Dup}^{(id,F)}.add(tag_i^{(id,F)})$ ;
Return  $D_{Nodup}^{(id,F)}, D_{Dup}^{(id,F)}$ .

```

the validity of Δ by checking $e(E, pk_c^\Psi) = e(H(q_0 \cdot \dots \cdot q_{N-1}) \cdot I^{h_3(\Psi||Time_0)}, g)$. If it holds, id accepts Δ , otherwise, id rejects it.

- c) id produces signing private key $sk_0^{(id)} = \beta \in Z_p^*$ and computes corresponding public key $pk_0^{(id)} = g^\beta$. Simultaneously, id generates a deduplication key $sk_1^{(id)} = \alpha$ and corresponding deduplication public key $pk_1^{(id)} = g^{\alpha^{-1}}$.
- d) id randomly selects $b_i, c_i \in Z_p^*$, computes $a_i = b_i f_0(c_i || id, i) \prod_{j=0}^{i-1} q_j \pmod{\Psi}$, $i \in [1, N-1]$, and constructs a secret polynomial $f(x) = a_0 + \sum_{i=1}^{N-1} a_i x^i$ of degree $N-1$, according to Chinese Remainder Theorem, $a_0 \in Z_\Psi^*$ satisfying $\{a_0 = sk_0 \pmod{q_i}\}_{i \in [1, N-1]}$.
- e) id computes the verification set $\{g^{a_i}\}_{i \in [0, N-1]}$, produces the shared key $sk_{1,t}^{(id)} = f(t)$, and resignature trapdoor key $y_t = \oplus [sk_{1,i}^{(id)}]_{i \in [1, N], i \neq t}$ for corresponding proxy P_t , $t \in [1, N]$.
- f) Upon receiving $sk_{1,t}^{(id)}$, each proxy P_t verifies whether $g^{sk_{1,t}^{(id)}} = \prod_{i=0}^{N-1} g^{a_i t^i}$ holds, determines the legitimacy of $sk_{1,t}^{(id)}$.

Similarly, id could execute the same steps above to share $sk_0^{-1} = \beta^{-1}$ with N proxies, and each proxy P_t , $t \in [1, N]$ will receive the shared key value $sk_{2,t}^{(id)}$. Essentially, the construction of polynomials and the generation of shared key can be achieved in offline way.

B. Duplication Detection

For a data file F to be outsourced to cloud storage systems, BA-FGDIA achieves duplication detection as follows.

- 1) A user id first splits the outsourced data file F with $Tag^{(id,F)} \in Z_p^*$ into n blocks as $\vec{F} = \langle m_1, m_2, \dots, m_n \rangle$. For each data block m_i , id computes partial deduplication tag $tag_i^{(id,F)} = pk_c^{sk_1^{(id)} + sk_1^{(id)} h_3(Enc(h_5(m_i), m_i))}$, $i \in [1, n]$ associated with corresponding ciphertext, obtains the deduplication tag set $DB^{(id,F)} = \{tag_i^{(id,F)}\}_{i \in [1, n]}$, and sends $\{Tag^{(id,F)}, DB^{(id,F)}\}$ to CSP.
- 2) Upon receiving $\{Tag^{(id,F)}, DB^{(id,F)}\}$, CSP computes the final deduplication tag $tag_i^{(id,F)}$ based on $tag_i^{(id,F)}$, gets nonduplication set $DB_{Nodup}^{(id,F)}$ and the duplication set $DB_{Dup}^{(id,F)}$ based on Algorithm 1, to determine whether $tag_i^{(id,F)} \in DB^{(id,F)}$ has identical data blocks with $DB = \{DB^{(id^*,F^*)}\}$ maintained by CSP.
- 3) CSP first gets an ordered set of detection result set $\{V_{re}\}$, where $|V_{re}| = n$, and

$$re = \begin{cases} 0X, \text{ inter} & V_{0X} = \begin{cases} 0, & X = 0 \\ 1, & X = 1 \end{cases} \\ 1X, \text{ exter} & \end{cases}$$

$$V_{1X} = \begin{cases} 0, & X = 0 \\ Tag^{(id^*,F^*)}, & X = 1. \end{cases}$$

Then, CSP generates an encrypted data reconstruction matrix $\mathbb{M} = [e_{ij}]_{|V_{re}| \times |V_{re}|}$ based on $\{V_{re}\}$, such that

$$e_{ij} = \begin{cases} \{0/Tag^{(id^*,F^*)}\}, & i = j \\ \{0/1\}, & i \neq j, i \leq j, i, j \in [1, |V_{re}|] \\ 0, & \text{Otherwise.} \end{cases}$$

Finally, CSP compresses $\mathbb{M}$ into a reconstruction array $\tilde{\mathbb{M}}$, such that $\tilde{\mathbb{M}}[k] = e_{jj}, k = [(i-1) \times (2n-i+2) + 2(j-i)/2]$, stores $\tilde{\mathbb{M}}$ in the cloud server and sends a detection result bit-string R to id . Here, R is generated according to $\tilde{\mathbb{M}}$, if $\tilde{\mathbb{M}}[k] = 0$, $R[k] = 0$; otherwise, $R[k] = 1, k \in [0, \tilde{\mathbb{M}}]$.

- 4) id first reconstructs a unimodular matrix $\mathbb{R}_{n \times n}$, based on the detection result R , id computes

$$\mathbb{R}[ij] = \begin{cases} e'_{ij} = R[k], & k = \frac{(i-1) \times (2n-i+2) + 2(j-i)}{2}, i \leq j \\ e'_{ij} = 0, & i \geq j, i, j \in [1, n]. \end{cases}$$

id computes the deduplication result vector $\vec{F} = \vec{F}^T \cdot \mathbb{R}$ to determine whether there exists internal or external duplication. If $\vec{F} = \mathbf{0}$, all data blocks are the original data blocks. If $\vec{F} \neq \mathbf{0}$, $\vec{F}[i] = 0$ denotes the nonduplicate data block, $\vec{F}[i] = \vec{F}[i]$ indicates m_i as duplicate data block that externally exists in another file F^* in the cloud server, $\vec{F}[i] = \vec{F}[j], i \neq j, i, j \in [1, n]$ represents $m_i = m_j$ within the data file F .

Finally, id records the number of internal duplicate data blocks C_{in} and external duplication data blocks C_{ex} , and sets a 2-bit status marker of the data file F as below. label = 00 indicates F is the original file; label = 01 indicates data blocks in the file have internal duplication; label = 10 indicates the file has external duplication in which there has existed data block backups in the cloud

server; label = 11 indicates there are internal and external duplication of data blocks in F . Then, id stores the deduplication credential $cred = \text{Tag}^{(id,F)} || \text{label} || c_{in} || c_{ex}$.

C. Initial Uploading

If $\vec{F} = \mathbf{0}$ and label = 00, it indicates F is an original data file in which no backup exists for each data block in the cloud server. The detailed steps are described as follows.

- 1) id randomly selects $r_i^{(id,F)} \in Z_p^*$, computes $\mu_i^{(id,F)} = h_2(g^{r_i^{(id,F)}} \cdot g^{h_5(\text{Tag}^{(id,F)}), h_3(m_i)})$, and $g^{r_i^{(id,F)} h_3(m_i)^{-1}}$, obtains an encryption key $key_i = h_5(\mu_i^{(id,F)})$ for each data block $m_i, i \in [1, n]$. id further computes each ciphertext block $c_i = \text{Enc}(key_i, m_i), i \in [1, n]$, and obtains the ciphertext set $C = \{c_1, c_2, \dots, c_n\}$. Then id computes integrity auditing auxiliary information $\varkappa = \{h_3(id || \text{Tag}^{(id,F)}), \{h_3(c_i)\}_{i \in [1, n]}\}$ and puts it on the blockchain.
- 2) id generates a signature set $\sigma = \{\sigma_i\}_{i \in [1, n]}$ by computing the homomorphic signature $\sigma_i = (H(c_i + ih_3(c_i) \cdot \text{Tag}^{(id,F)}) \cdot W^{c_i})^{sk_0^{(id)}}$ for each $c_i, i \in [1, n]$. id sends $\{\text{Tag}^{(id,F)}, C, \sigma, \phi\}$ to CSP, where $\phi = \{g^{r_i^{(id,F)} h_3(m_i)^{-1}}\}$ is a sharing set of partial auxiliary encryption key.
- 3) Upon receiving $\{\text{Tag}^{(id,F)}, C, \sigma, \phi\}$, CSP verifies whether $e(\prod_{i=1}^n \sigma_i, g) = e(H(\sum_{i=1}^n c_i + \text{Tag}^{(id,F)} \sum_{i=1}^n h_3(c_i) i) \cdot W^{\sum_{i=1}^n c_i}, pk_0^{(id)})$ holds. If it does not hold, CSP rejects to store $\{\sigma, C\}$. Otherwise, CSP creates a dynamic ciphertext tree SD with size $\mathcal{T}_{N,d}$ based on simplified $SSVT$ in Algorithm 2, and stores $\{id, \text{label} = 00, \text{Tag}^{(id,F)}, D_{\text{Nodup}}^{(id,F)}, \sigma, SD, \phi, \tilde{M}\}$ in the cloud server.

D. Subsequent Uploading

If $\vec{F} \neq \mathbf{0}$ and label = 1X, there are external duplicated data blocks in $F = m_1 || m_2 || \dots || m_n$. id and CSP execute PoW protocol to determine the ownership of data blocks as below.

- 1) Based on *DuplicationDetection*, CSP gets the duplicate tag set $DB_{\text{Dup}}^{(id,F)}$, and computes the shared auxiliary encryption key set $\{g^{h_5(\text{Tag}^{(id_l, F_{l_i})})}, g^{r_{l_i}^{(id_l, F_{l_i})} h_3(m_{l_i})^{-1}}\}_{l_i \in [1, |DB_{\text{Dup}}|]}$ for duplicate data blocks. CSP sends the partial encryption key set $\mathcal{B} = \{\{g^{h_5(\text{Tag}^{(id_l, F_{l_i})})}, g^{r_{l_i}^{(id_l, F_{l_i})} h_3(m_{l_i})^{-1}}\}\}$ to id .
- 2) id finds the data block m_j such that $\vec{F}[j] = \vec{F}[j], j \in [1, n]$ holds, obtains corresponding partial key $\{g^{h_5(\text{Tag}^{(id^*, F^*)})}, g^{r_{l_j}^{(id^*, F^*)} h_3(m_j')^{-1}}\}$ from $\mathcal{B}$, computes an encryption key $key'_j = h_2(g^{h_3(m_j')^{-1} r_{l_j}^{(id^*, F^*)} h_3(m_j)}, g^{h_5(\text{Tag}^{(id^*, F^*)}), h_3(m_j)})$, and obtains the duplicate ciphertext $c_j = \text{Enc}(key'_j, m_j)$. Finally, id sends the duplicate ciphertext set $C_{\text{Dup}} = \{c_{l_1}, c_{l_2}, \dots, c_{l_i}\}_{l_i \in [1, |\mathcal{B}| \ll n]}$ to CSP.
- 3) CSP identifies the ciphertext tree SD^* with backup ciphertext c_j^* corresponded to the ciphertext c_{l_i} for the duplicate tag $\text{tag}_{l_i}^{(id,F)}$ in $DB_{\text{Dup}}^{(id,F)}$, calculates the opening value of corresponding vector commitment V at the ciphertext position $j, j \in [0, N-1]$, and aggregates the

Algorithm 2 Generation of Ciphertext Tree

Require: κ_2, sk_c , node degree $\mathbb{N}$, tree depth $d, DB_{\text{Nodup}}^{(id,F)}$, and ciphertext set C .

Ensure: Ciphertext tree SD .

```

 $\varpi = \{\gamma_0, \dots, \gamma_{\mathbb{N}}, \gamma_{\mathbb{N}+2}, \dots, \gamma_{2\mathbb{N}+1}\} : PPGen(1^{\kappa_2}, \mathbb{N}, d, sk_c),$ 
 $\gamma_j = g^{sk_c^{j+1}};$ 
 $SD = [0], \varepsilon = NULL;$ 
 $\zeta \in Z_p^*;$ 
 $\varepsilon \leftarrow \varepsilon \cup h_4(\text{tag}_i^{(id,F)} || i), i \in [1, n] \& \& \text{tag}_i^{(id,F)} \in DB_{\text{Nodup}}^{(id,F)};$ 
For  $\mathcal{L} \in SetupTree(\varepsilon)$  do
    if  $\mathcal{L} = \mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_d$  then;
         $SD[\mathcal{L}] = 0;$ 
    else
         $V = \text{commit}(\varpi, 0^{\mathbb{N}+1}, \zeta) : \varpi[:\mathbb{N} + 1]^{\mathbf{v} || \zeta} = \gamma_{\mathbb{N}}^{\zeta}.$ 
         $\prod_{j=0}^{\mathbb{N}-1} \gamma_j^{v_j};$ 
         $SD[\mathcal{L}] = \langle V, 0^{\mathbb{N}+1}, \zeta \rangle;$ 
For  $\mathcal{L} \in Tree(\varepsilon)$  do
    if  $\mathcal{L} = \mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_d \& \& \mathcal{L} = h_4(\text{tag}_i^{(id,F)} || i)$  then;
         $SD[\mathcal{L}] = c_i;$ 
    else if  $\mathcal{L} = \mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_{d-1}$  then;
        for  $j \in [0, \mathbb{N} - 1]$  do
             $v_j = SD[\mathcal{L} || j];$ 
             $\mathbf{v} = \langle v_1, v_2, \dots, v_n, \sum_{j=0}^{\mathbb{N}-1} v_j \rangle;$ 
             $V = \text{commit}(\varpi, \mathbf{v}, \zeta) : \varpi[:\mathbb{N} + 1]^{\mathbf{v} || \zeta} = \gamma_{\mathbb{N}}^{\zeta}.$ 
             $\prod_{j=0}^{\mathbb{N}-1} \gamma_j^{v_j};$ 
             $SD[\mathcal{L}] = \langle V, \mathbf{v}, \zeta \rangle;$ 
        else
            for  $j \in [0, \mathbb{N} - 1]$  do
                 $\langle V_j, \mathbf{v}_j, \zeta \rangle \leftarrow SD[\mathcal{L} || j];$ 
                 $v_j = \mathbf{v}_j[\mathbb{N}];$ 
                 $\mathbf{v} = \langle v_1, v_2, \dots, v_n, \sum_{j=0}^{\mathbb{N}-1} v_j \rangle;$ 
                 $V = \text{commit}(\varpi, \mathbf{v}, \zeta) : \varpi[:\mathbb{N} + 1]^{\mathbf{v} || \zeta} = \gamma_{\mathbb{N}}^{\zeta}.$ 
                 $\prod_{j=0}^{\mathbb{N}-1} \gamma_j^{v_j};$ 
                 $SD[\mathcal{L}] = \langle V, \mathbf{v}, \zeta \rangle;$ 
    Return  $SD$ .
```

opening value of the duplicate ciphertext. The specific steps are described as follows.

- a) Suppose the backup of c_{l_i} corresponded to $\text{tag}_{l_i}^{(id,F)} \in DB_{\text{Dup}}^{(id,F)}, l_i \in [1, |DB_{\text{Dup}}^{(id,F)}|]$, which is distributed among η vector commitments $\{V_j\}_{j \in [1, \eta]}$ of different ciphertext trees SD . Set $\omega_j \subset [0, \mathbb{N} - 1]$ to be the index subset of the opening value in V_j .
- b) CSP computes the opening value of a single position $i, i \in \omega_j$ in vector commitment V_j as $\Phi = ([V_j / \gamma_i^{v_{ji}}])^{sk_c^{N+1-i}}$, and computes all opening values $\{\Phi_i\}_{i \in \omega_j}$ in V_j , further aggregates $|\omega_j|$ opening values $\hat{\Phi}_j = \prod_{i \in \omega_j} \Phi_{ji}^{x_{ji}}, x_{ji} = h_3(i, V_j, \omega_j, \mathbf{v}_j[\omega_j])$ and $\sum_{j=1}^{\eta} |\omega_j|$ opening values $\bar{\Phi} = \prod_{j=1}^{\eta} \hat{\Phi}_j^{x'_j}, x'_j = h_3(j, \{V_j, \omega_j, \mathbf{v}_j[\omega_j]\}_{j \in [1, \eta]})$ in vector commitment $\{V_j\}_{j \in [1, \eta]}$.
- 4) Upon receiving $C_{\text{Dup}} = \{c_{l_1}, c_{l_2}, \dots, c_{l_i}\}$, CSP checks $\prod_{j=1}^{\eta} e(V_j^{x'_j}, \prod_{i \in \omega_j} \gamma_{N-i}^{x_{ji}}) = e(\bar{\Phi}, g)$.

$e(\gamma_0, \gamma_N) \sum_{j=1}^n x_j' \sum_{i \in \omega_j} c_i x_{ji}$ based on C_{Dup} and $\bar{\Phi}$, where $c_i \in C_{Dup}$ and $|DB_{Dup}^{(id,F)}| = \sum_{j=1}^n |\omega_j|$. If it holds, it indicates that a user does own these data blocks and sends $Result_0 = 1$ to id . Then, CSP generates a searchable duplicate location index trap $T_{li} = h_3(index^s || Tag^{(id,F)} || id || index^t || tag_{index^t}^{(id^*, F^*)})$ for each $c_{li} \in C_{Dup}$ and updates count bloom filter $CBF_{(w,l)}.insert(T_{li})$, where $index^s$ represents the index value of c_i in F , and $index^t$ represents the index value in $Tag^{(id^*, F^*)}$ of c_i 's backup.

- 5) After id receives the feedback $Result_0$, the unduplicate data blocks $\{m_j\}_{j \in [1,n], \vec{F}[j]=0}$ in F are uploaded to the cloud server by following the steps of *initial uploading*. Finally, CSP stores $\{id, label = 10/11, Tag^{(id,F)}, DB_{Nodup}^{(id,F)}, \sigma, SD, \phi, \tilde{M}\}$, and integrity auditing auxiliary information $\varkappa$ are put on blockchain.

Remark: We do not discuss the case of $\vec{F} \neq \mathbf{0}$ and $label = 01$, since when there exists internal duplication, id only needs to manually upload a backup for identical data blocks m_i and m_j , and does not execute the PoW protocol.

E. Integrity Auditing

Integrity auditing includes *Challenge Generation*, *Proof Generation*, and *Verification*.

Challenge Generation: When the status marker of $Tag^{(id,F)}$ is not $label = 00$, id sends an integrity auditing request $\{Token || r^{-1}, Tag^{(id,F)}\}$ to trigger smart contract SC_1 to generate the challenge information $Chal^{(id,F)}$, where $Token = Enc(\oplus[sk_{1,t}^{(id)}]_{t \in [1,N]}, r || Time_2)$ is a resignation token, $Time_2$ is the current timestamp and $r \in Z_p^*$. Otherwise, id sends an integrity auditing request $Tag^{(id,F)}$. As such, SC_1 can obtain latest t block hashes $\{B_1, B_2, \dots, B_t\}$ of blockchain, generate two random seeds $\theta_1 = h_3(r^{-1} || Tag^{(id,F)} || B_1 || B_2 || \dots || B_t || Time_3 || 1)$, $\theta_2 = h_3(r^{-1} || Time_3 || B_1 || B_2 || \dots || B_t || Tag^{(id,F)} || 2)$, and the number of challenge data blocks $\lambda \in [1, n]$. SC_1 puts $Chal^{(id,F)} = \{h_3(Tag^{(id,F)}), Token, \theta_1, \theta_2, \lambda\}$ on the blockchain.

Proof Generation: Upon retrieving $Chal^{(id,F)}$ from the blockchain, CSP computes $\{\delta_i, \vartheta_i\}_{i \in [1,\lambda]}$, where $\delta_i = \pi_0(i, \theta_1)$ and $\vartheta_i = f_1(i, \theta_2)$, then generates integrity auditing proof information *Proof* based on status marker $label$.

- 1) If $label = 00$, CSP generates $Proof = \{\psi, \varphi\}$ as follows:

$$\psi = \prod_{i=1}^{\lambda} \sigma_{\delta_i}^{\vartheta_i}, \varphi = \sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}.$$

- 2) If $label = 01$, it indicates there may exist an internal duplicate challenge data block index subset $\{\delta_j', \vartheta_j'\}_{j \in [1,\lambda]} \subseteq \{\delta_i, \vartheta_i\}_{i \in [1,\lambda]}$, where corresponding leaf node value in the ciphertext tree SD is 0. CSP generates $Proof = \{\psi_0, \psi_1, \varphi\}$ by executing the following steps for challenge index δ_j' .

- a) Based on $\{\delta_j', \vartheta_j'\}_{j \in [1,\lambda]}$ and the reconstruction array $\tilde{M}$, CSP obtains backup data block $c_\epsilon \leftarrow SD[tag_\epsilon^{(id,F)} || \epsilon]$, $\epsilon \in [1, n] / \{\delta_j'\}_{j \in [1,\lambda]}$ and the signature σ_ϵ of $c_{\delta_j'}$ in data file with

$Tag^{(id,F)}$, and the number of internal duplicate blocks as c_{in}' , CSP sets $c_{\delta_j'} = c_\epsilon$.

- b) CSP obtains the proof information about number of deduplicated data blocks $VI = \{\Lambda, \Upsilon, K\}$, where CSP calculates $u = h_1(c_{in}' || \theta_1 || \theta_2 || Tag^{(id,F)} || label)$, and randomly chooses $\Upsilon_0 \in Z_p^*$, such that $\widehat{\Upsilon_0} = \Gamma(u)$. Let Υ_0 do $\mathbf{I}$ iterations $\Upsilon_i = \tau(\Upsilon_{i-1})_{i \in [1,\mathbf{I}]}$ to get $\Upsilon = \Upsilon_{\mathbf{I}}$ and $\Lambda = h_1(\Gamma^{-1}(\widehat{\Upsilon_{\mathbf{I}}}))$. CSP generates a verifiable threshold trap $K = SH(\Upsilon_{\mathbf{I}}, label || c_{in}')$ and obtains q_{K-1} .
- c) CSP generates the auxiliary parameter $H(Tag^{(id,F)} \delta_j' h_3(c_\epsilon) - Tag^{(id,F)} \epsilon h_3(c_\epsilon))^{r_0}$ for $c_{\delta_j'}$, sends resignation request $\{id, Token, q_{K-1}, H(Tag^{(id,F)} \delta_j' h_3(c_\epsilon) - Tag^{(id,F)} \epsilon h_3(c_\epsilon))^{r_0}, \sigma_\epsilon^{r_0}\}$ to N proxies, where $r_0 \in Z_p^*$ is randomly chosen.
- d) Each proxy $P_t, t \in [1, N]$ first decrypts $Token$ to obtain $r || Time_2 = Enc(y_t \oplus sk_{1,t}^{(id)}, Token)$, verifies whether $Time_4 - Time_2 \leq DT$ holds. If so, it computes $A_{\delta_j'}^t = H(Tag^{(id,F)} \delta_j' h_3(c_\epsilon) - Tag^{(id,F)} \epsilon h_3(c_\epsilon))^{r_0 r (sk_{1,t}^{(id)}) \pmod{q_{K-1}}}$ and $\sigma_{\epsilon,t}^{r_0 r}$. Each proxy $P_t, t \in [1, N]$ sends $A_{\delta_j'}^t$ and $\sigma_{\epsilon,t}^{r_0 r}$ to CSP. Otherwise, rejects it.
- e) Upon receiving $\{A_{\delta_j'}^t\}_{t \in [1,K]}$, CSP computes the lagrange interpolation coefficient $\{\xi_{0,t} = \prod_{j \in [1,K], t \neq j} (j/j - t)\}_{t \in [1,K]}$ and partial random authentication tag $A_{\delta_j'} = \prod_{t=1}^K (A_{\delta_j'}^t)^{\xi_{0,t}}$, then generates

Proof:

$$\begin{aligned} \psi_0 &= \prod_{\delta_i \in \{\delta_i, \vartheta_i\} / \{\delta_j', \vartheta_j'\}} \sigma_{\delta_i}^{\vartheta_i} \\ \psi_1 &= \prod_{\delta_j' \in \{\delta_j', \vartheta_j'\}} (\sigma_{\epsilon}^{r_0 r} A_{\delta_j'}^t)^{r_0^{-1} \vartheta_j'} \\ \varphi &= \sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}, i \in [1, \lambda]. \end{aligned}$$

- 3) If $label = 10$, there may exist an external duplicate challenge data block index subset $\{\delta_j', \vartheta_j'\}_{j \in [1,\lambda]} \subseteq \{\delta_i, \vartheta_i\}_{i \in [1,\lambda]}$, where corresponding ciphertext is in other outsourced data file with $Tag^{(id^*, F^*)}$. CSP generates $Proof = \{\psi_0, \psi_1, \varphi\}$ by executing the following steps for challenge index δ_j' .

- a) CSP locates the data file with $Tag^{(id^*, F^*)}$ including the backup ciphertext of $c_{\delta_j'}$ based on $\tilde{M}$. Then, CSP computes the searchable duplicated location trap set $\{T_\chi = (h_3(\delta_j' || Tag^{(id,F)} || id || \chi || tag_\chi^{(id^*, F^*)}))\}$ by traversing $tag_\chi^{(id^*, F^*)}, tag_\chi^{(id^*, F^*)} \in DB_{Nodup}^{(id^*, F^*)} || \chi \in [1, n] || M^*[\chi \chi] = 0$, based on $\tilde{M}^*$ for $Tag^{(id^*, F^*)}$, and obtains the data block with $tag_\chi^{(id^*, F^*)}$ satisfying $CBF_{(w,l)}.check(T_\chi) \neq 0$. CSP sets $c_{\delta_j'} \leftarrow c_\chi^{F^*} = SD^{F^*}(tag_\chi^{(id^*, F^*)} || \chi)$, and gets signature $\sigma_\chi^{F^*}$ of $c_\chi^{F^*}$.

- b) CSP gets the number of external duplicate data blocks C'_{ex} based on $\tilde{M}$, computes $u = h_1(c'_{ex}||\theta_1||\theta_2||\text{Tag}^{(id,F)}||\text{label})$, and generates the proof information about number of deduplicated data blocks $VI = \{\Lambda, \Upsilon, K\}$.
- c) CSP generates the auxiliary parameter $H(\text{Tag}^{(id,F)}\delta'_j h_3(c_{\chi}^{F*}) - \text{Tag}^{(id*,F*)}\chi h_3(c_{\chi}^{F*}))^{r_0}$ for $c_{\delta'_j}$ in $\{\delta'_j, \vartheta'_j\}_{j \in [1, \lambda]}$, and sends $\{id \rightarrow id^*, \text{Token}, q_{K-1}, H(\text{Tag}^{(id,F)}\delta'_j h_3(c_{\chi}^{F*}) - \text{Tag}^{(id*,F*)}\chi h_3(c_{\chi}^{F*}))^{r_0}, \sigma_{\chi}^{(F*)^{r_0}}\}$ to N proxies to get the signature $\sigma_{\delta'_j}$, where $r_0 \in Z_p^*$ is randomly chosen.
- d) Each proxy $P_t, t \in [1, N]$ first decrypts Token to obtain $r||\text{Time}_2 = \text{Enc}(y_t \oplus sk_{1,t}^{(id)}, \text{Token})$, verifies whether $\text{Time}_4 - \text{Time}_2 \leq DT$ holds. If so, each proxy $P_t, t \in [1, N]$ computes $\sigma_{\delta'_j,1}^t = H(\text{Tag}^{(id,F)}\delta'_j h_3(c_{\chi}^{F*}) - \text{Tag}^{(id*,F*)}\chi h_3(c_{\chi}^{F*}))^{r_0 r(sk_{1,t}^{(id)} \pmod{q_{K-1}})}$.
- e) To get $\sigma_{\delta'_j,2}$, each proxy $P_t, t \in [1, N]$ computes $\Omega_{0,t} = sk_{2,t}^{(id*)} \cdot sk_{1,t}^{(id)} \pmod{q_{K-1}}$, randomly chooses $a_i \in Z_p^*, i \in [1, K]$, and constructs a polynomial $f_t(x) = \Omega_{0,t} + \sum_{i=1}^{K-1} a_i x^i$ of order $K-1$. Then, each proxy $P_t, t \in [1, N]$ computes $S_{t,j} = f_t(j), t \neq j, j \in [1, N]$ and sends $S_{t,j}$ to corresponding proxy P_j .
- f) Each proxy $P_t, t \in [1, N]$ computes a recombination vector $(e_1, e_2, \dots, e_N)$. Upon receiving $\{S_{j,t}, j \neq t\}$ from another proxies, each proxy P_t computes $S_t = \sum_{j=1}^N e_j S_{j,t}$ and $\sigma_{\delta'_j,2}^t = (\sigma_{\chi}^{F*})^{r r_0 S_t}$, sends $\sigma_{\delta'_j}^t = \sigma_{\delta'_j,1}^t \cdot \sigma_{\delta'_j,2}^t$ to CSP.
- g) Upon receiving $\sigma_{\delta'_j}^t$, CSP computes $\sigma_{\delta'_j} = \prod_{t=1}^K (\sigma_{\delta'_j}^t)^{\xi_{0,t}}$, and generates *Proof* as follows.

$$\begin{aligned} \psi_0 &= \prod_{\delta_i \in \{\delta_i, \vartheta_i\} / \{\delta'_j, \vartheta'_j\}} \sigma_{\delta_i}^{\vartheta_i} \\ \psi_1 &= \prod_{\delta'_j \in \{\delta'_j, \vartheta'_j\}} (\sigma_{\delta'_j}^t)^{r_0^{-1} \vartheta'_j} \\ \varphi &= \sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}, i \in [1, \lambda]. \end{aligned}$$

- 4) If $\text{label} = 11$, CSP generates *Proof* by following the steps of $\text{label} = 01$ and $\text{label} = 10$. However, note that when generating *VI* to prevent malicious deduplication of CSP and the threshold trap K , the parameters are calculated as $u = h_1(c'_{in}||c'_{ex}||\theta_1||\theta_2||\text{Tag}^{(id,F)}||\text{label})$ and $K = SH(\Upsilon_1, \text{label}||c'_{in}||c'_{ex})$.

Verification: When CSP generates *Proof* = $\{\psi, \varphi\}$ or *Proof* = $\{\psi_0, \psi_1, \varphi\}$, smart contract SC_2 depicted in Algorithm 3 is triggered for integrity verification as follows.

$$e(\Theta, g) = e(H(\varphi + \Pi) \cdot W^{\varphi}, pk_0^{(id)}). \quad (1)$$

Algorithm 3 Smart Contract 2 in Integrity Auditing

Require: *Proof* = $\{\psi, \varphi\}$ or *Proof* = $\{\psi_0, \psi_1, \varphi\}$, *Chal*^(id,F), auxiliary information *VI* and $\varkappa$ on the blockchain.

Ensure: The result *isIntegrity* of verification equation

```

Result1 = False;
log(id,F) = NULL;
SC1 = contract(SC1Address);
r-1 = SC1(parseValue(Token)||r-1);
ω = {(\delta_i, \vartheta_i)} = SC1(getValue);
Π = 0;
for each i ∈ [1, λ = ω.length] && h3(c\delta_i) ∈ \varkappa do
    Π = Π + Tag(id,F) · \delta_i · ω[\delta_i] · h3(c\delta_i);
if label = 00 then
    Θ = \psi;
if label ≠ 00 then
    Θ = \psi_0 · \psi_1^{r^{-1}};
If e(Θ, pk_0^{(id)}) = e(H(\varphi + \Pi) · W^{\varphi}, g) then
    Result1 = True;
log(id,F) = {h3(Tag(id,F)), VI, Proof, Result1};
%Put the log information on the blockchain.
Outputs.push(log(id,F));
Return Result1.

```

SC_2 produces an integrity auditing log information $\log^{(id,F)} = \{h_3(\text{Tag}^{(id,F)}), VI, \text{Proof}, \text{Result}_1\}$ and puts it on the blockchain.

F. Log Checking

When *id* obtains the integrity auditing result from the blockchain, the log information $\log^{(id,F)} = \{h_3(\text{Tag}^{(id,F)}), VI, \text{Proof}, \text{Result}_1\}$ and $\text{Chal}^{(id,F)} = \{h_3(\text{Tag}^{(id,F)}), \text{Token}, \theta_1, \theta_2, \lambda\}$ are extracted. *id* judges whether CSP has malicious deduplication behaviors according to $VI = \{\Lambda, \Upsilon, K\}$, and deduplication credential $\text{cred} = \text{Tag}^{(id,F)}||\text{label}||c_{in}||c_{ex}$ stored by himself/herself through the following:

$$\begin{aligned} u' &= h_1 \left(X = \begin{cases} c_{in}, & \text{label} = 01 \\ c_{ex}, & \text{label} = 10 \quad ||\theta_1||\theta_2||\text{Tag}^{(id,F)}||\text{label} \\ c_{in}||c_{ex} & \text{label} = 11, \end{cases} \right) \\ \Lambda &= h_1 \left(\Gamma^{-1}(\Upsilon); \Gamma((\tau^{-1})^I(\Upsilon)) \right) = \Gamma(u') \\ K &= SH \left(\Upsilon, \text{label}||X = \begin{cases} c_{in}, & \text{label} = 01 \\ c_{ex}, & \text{label} = 10 \\ c_{in}||c_{ex} & \text{label} = 11 \end{cases} \right). \end{aligned}$$

If one of the equations does not hold, it indicates CSP has maliciously deleted duplicate data blocks. Otherwise, *id* views the integrity auditing Result_1 . *id* can choose whether to execute an integrity auditing by himself/herself according to $\text{Proof} = \{\psi_0, \psi_1, \varphi\}$, $\text{Chal}^{(id,F)} = \{h_3(\text{Tag}^{(id,F)}), \theta_1, \theta_2, \lambda\}$ and $\varkappa = \{h_3(id||\text{Tag}^{(id,F)}), \{h_3(c_i)\}_{i \in [1, n]}\}$, to check $e(\psi_0 \cdot \psi_1^{(r^{-1})}, g) = e(H(\varphi + \text{Tag}^{(id,F)} \sum_{i=1}^{\lambda} h_3(c_{\delta_i}) \delta_i \vartheta_i) \cdot W^{\varphi}, pk_0^{(id)})$.

G. Data Retrieval

Upon checking the integrity auditing log information $\log^{(id,F)}$, if $\text{Result}_1 = 1$, *id* can download the data file with

$\text{Tag}^{(id,F)}$. Now we focus on showing an example about the recovery of the data file F with $\text{Tag}^{(id,F)}$ which has been deduplicated by CSP.

- 1) CSP recovers the ciphertext reconstruction matrix $\mathbb{M}$ from $\bar{\mathbb{M}}$, obtains $C_F = \langle c_1, 0, 0, 0, \dots, c_{n-4}, 0, c_{n-2}, c_{n-1}, 0 \rangle$ based on the ciphertext tree SD and duplicate tag $DB_{\text{Nodup}}^{(id,F)}$. CSP computes partial internal deduplicated data blocks by $C'_F = C_F + C_F^T \cdot \mathbb{M} = \langle c_1, 0, 0, c_1, \dots, c_{n-4}, c_{10}, c_{n-2}, c_{n-1}, 0 \rangle$.
- 2) CSP gets the location of the backup ciphertext of the external duplicate block $C'_F[j]$ which satisfies $C'_F[j] = 0$, and $\mathbb{M}[j] \neq 0$, which is the data file with $\text{Tag}^{(id^*, F^*)}$ based on $\mathbb{M}$, then CSP obtains $c_{\chi}^{F^*}$ that satisfies $CBF_{(w,i)}.\text{check}(h_3(j||\text{Tag}^{(id,F)}||id||\chi||\text{tag}_{\chi})) \neq 0, \chi \in [1, n]||\text{tag}_{\chi} \in DB_{\text{Nodup}}^{(id^*, F^*)}$. CSP sets $C'_F[j] = c_{\chi}^{F^*}$.
- 3) CSP obtains the backup ciphertext of all external duplicate blocks. In this case, C'_F is updated to $C''_F = \langle c_1, c_{\chi}^{F^*}, 0, c_1, \dots, c_{n-4}, c_{10}, c_{n-2}, c_{n-1}, c_{\chi'}^{F^*} \rangle$. Furthermore, CSP obtains the ciphertext $C = \langle c_1, c_{\chi}^{F^*}, c_{\chi'}^{F^*}, c_1, \dots, c_{n-4}, c_{10}, c_{n-2}, c_{n-1}, c_{\chi'}^{F^*} \rangle$ based on $\mathbb{M}$.
- 4) CSP sends C to id . id obtains the auxiliary information $\varkappa = \{h_3(id||\text{Tag}^{(id,F)}), \{h_3(c_i)\}_{i \in [1, n]}\}$ from blockchain and verifies whether $\sum_{i=1}^n h_3(c'_i) = \sum_{i=1}^n h_3(c_i)$, $c'_i \in C$ holds. If so, id decrypts the ciphertext C to retrieve the data file $F = m_1||m_2||\dots||m_n$. Thus, id succeeds in downloading and retrieving the data file F .

Actually, BA-FGDIA also achieves batch integrity auditing. To be specific, id sends the auditing request $\{\{\text{Tag}_i^{(id,F)}\}_{i \in [1, k]}, \text{Token}||r^{-1}\}$ to trigger the smart contract SC_1 . Then, SC_1 computes challenge information $\text{Chal} = \{\{h_3(\text{Tag}_i^{(id,F)})\}, \text{Token}, \theta_1, \theta_2, \lambda\}$ and sends Chal to CSP. Upon receiving Chal , CSP generates $\text{Proof} = \{\psi_0, \psi_1, \varphi\}$ as before, satisfying $\psi_0 = \prod_{i=1}^k \psi_{i,0}$, $\psi_1 = \prod_{i=1}^k \psi_{i,1}$ and $\varphi = \sum_{i=1}^k \varphi_i$. Finally, CSP sends Proof to trigger smart contract SC_2 to verify $e(\Theta, pk_0^{(id)}) = e(H(\varphi + \Pi) \cdot W^{\varphi}, g)$, where Π is precomputed by SC_2 . At last, SC_2 generates the auditing log $\log^{(id,F)} = \{h_3(\text{Tag}_1^{(id,F)}||\dots||\text{Tag}_k^{(id,F)}), \{VI_i\}_{i \in [1, k]}, \text{Proof}, \text{Result}_1\}$ on the blockchain.

H. Correctness Consistence of BA-FGDIA

Correctness of Deduplication: In *Duplication Detection*, id sends deduplication tag set $DB^{(id,F)}$ to be compared with tags in the cloud server according to $\text{tag}_i^{(id,F)} = \text{tag}_j^{(id^*, F^*)}$, which is elaborated as follows:

$$\begin{aligned} \text{tag}_i^{(id,F)} &= e\left(\text{tag}_i'^{(id,F)} \cdot pk_1^{(id)^{-1}} \cdot pk_1^{(id)}\right) \\ &= e\left(g^{-sk_1^{(id)^{-1}}} \cdot pk_c^{sk_1^{(id)^{-1}} + sk_1^{(id)} h_3(\text{Enc}(h_5(m_i), m_i))}\right) \\ &= e\left(g^{sk_1^{(id)^{-1}}} \cdot pk_c^{h_3(\text{Enc}(h_5(m_i), m_i))}, g\right) \\ \text{tag}_j^{(id^*, F^*)} &= e\left(\text{tag}_j^{(id^*, F^*)} \cdot pk_1^{(id^*)^{-1}} \cdot pk_1^{(id^*)}\right) \end{aligned}$$

$$\begin{aligned} &= e\left(g^{-sk_1^{(id^*)^{-1}}} \cdot pk_c^{sk_1^{(id^*)^{-1}} + sk_1^{(id^*)} h_3(\text{Enc}(h_5(m_j^*), m_j^*))}\right) \\ &= e\left(pk_c^{h_3(\text{Enc}(h_5(m_j^*), m_j^*))}, g\right). \end{aligned}$$

If $m_i = m_j^*$, there exists $\text{tag}_i^{(id,F)} = \text{tag}_j^{(id^*, F^*)}$ to determine duplication.

Then, CSP verifies the duplicate data ownership by $\prod_{j=1}^{\eta} e(V_j^{x'_j}, \prod_{i \in \omega_j} \gamma_{N-i}^{x_{ji}}) = e(\bar{\Phi}, g) \cdot e(\gamma_0, \gamma_N)^{\sum_{j=1}^{\eta} x'_j \sum_{i \in \omega_j} c_{ix_{ji}}}$, which is elaborated as follows:

$$\begin{aligned} \prod_{j=1}^{\eta} e\left(V_j^{x'_j}, \prod_{i \in \omega_j} \gamma_{N-i}^{x_{ji}}\right) &= \prod_{j=1}^{\eta} e\left(\prod_{i \in \omega_j} V_j^{x_{ji} x'_j sk_c^{N-i+1}}, g\right) \\ &= \prod_{j=1}^{\eta} e\left(\prod_{i \in \omega_j} \left(\frac{V_j}{\gamma_i}\right)^{x_{ji} x'_j sk_c^{N-i+1}} \cdot (\gamma_i^{v_{ji}})^{x_{ji} x'_j sk_c^{N-i+1}}, g\right) \\ &= \prod_{j=1}^{\eta} e\left(\prod_{i \in \omega_j} (\Phi_i)^{x_{ji} x'_j}, g\right) \cdot e\left(g^{sk_c^{N+2} \sum_{i \in \omega_j} v_{ji} x_{ji} x'_j}, g\right) \\ &= \prod_{j=1}^{\eta} e\left((\Phi_j)^{x'_j}, g\right) \cdot e(\gamma_0, \gamma_N)^{x'_j \sum_{i \in \omega_j} v_{ji} x_{ji}} \\ &= e\left(\prod_{j=1}^{\eta} (\hat{\Phi}_j)^{x'_j}, g\right) \cdot e(\gamma_0, \gamma_N)^{\sum_{j=1}^{\eta} x'_j \sum_{i \in \omega_j} v_{ji} x_{ji}} \\ &= e(\bar{\Phi}, g) \cdot e(\gamma_0, \gamma_N)^{\sum_{j=1}^{\eta} x'_j \sum_{i \in \omega_j} v_{ji} x_{ji}}. \end{aligned}$$

If id has the ciphertext set $\{v_{ji}\}_{j \in [1, \eta], i \in \omega_j}$ of these duplicate data blocks in the cloud server, the set of duplicate ciphertexts $C_{\text{Dup}} = \{c_{l_1}, c_{l_2}, \dots, c_{l_i}\}$ uploaded by id holds that $e(\gamma_0, \gamma_N)^{\sum_{j=1}^{\eta} x'_j \sum_{i \in \omega_j} v_{ji} x_{ji}} = e(\gamma_0, \gamma_N)^{\sum_{j=1}^{\eta} x'_j \sum_{i \in \omega_j} c_{ix_{ji}}}$, $c_i \in C_{\text{Dup}}$. Furthermore, this $\prod_{j=1}^{\eta} e(V_j^{x'_j}, \prod_{i \in \omega_j} \gamma_{N-i}^{x_{ji}}) = e(\bar{\Phi}, g) \cdot e(\gamma_0, \gamma_N)^{\sum_{j=1}^{\eta} x'_j \sum_{i \in \omega_j} c_{ix_{ji}}}$ holds.

Correctness of Integrity Auditing: We demonstrate the correctness of integrity auditing by taking the case of label = 01 as an example. Suppose that each proxy P_i has a polynomial of $f_i(x) = a_0 + \sum_{i=1}^{N-1} a_i x^i$, set $a_0 = (1, 0, 0, \dots, 0)(a_0, a_1, a_2, \dots, a_{N-1})^T$. If we want to share a_0 with N proxies, there exists a Vandermonde matrix M

$$M = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 2^1 & \dots & 2^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & N^1 & \dots & N^{N-1} \end{bmatrix}.$$

As there exists a unique recombination vector $(e_1, e_2, e_3, \dots, e_N) \in Z_p^*$ such that $(e_1, e_2, e_3, \dots, e_N)M = (1, 0, 0, \dots, 0)$, then

$$\begin{aligned} f(0) &= a_0 = (e_1, e_2, e_3, \dots, e_N)M(a_0, a_1, a_2, \dots, a_{N-1})^T \\ &= (e_1, e_2, e_3, \dots, e_N)(f(1), f(2), f(3), \dots, f(N))^T \\ &= \sum_{i=1}^N e_i f(i). \end{aligned}$$

Thus, we can further derive a $\sigma_{\delta'_j,2}$

$$\begin{aligned}
\sigma_{\delta'_j,2} &= \prod_{t=1}^K (\sigma_{\delta'_j,2}^t)^{r_0^{-1} \xi_{0,t}} = \prod_{t=1}^K ((\sigma_{\chi}^{F^*})^{r_{S_t}})^{\xi_{0,t}} \\
&= (\sigma_{\chi}^{F^*})^{r \cdot \sum_{t=1}^K S_t \xi_{0,t}} = (\sigma_{\chi}^{F^*})^{r \cdot \sum_{t=1}^K \xi_{0,t} \sum_{j=1}^N e_j S_{j,t}} \\
&= (\sigma_{\chi}^{F^*})^{r \cdot \sum_{j=1}^N e_j \sum_{t=1}^K \xi_{0,t} S_{j,t}} = (\sigma_{\chi}^{F^*})^{r \cdot \sum_{j=1}^N e_j \Omega_{0,j}} \\
&= (\sigma_{\chi}^{F^*})^{r \cdot \sum_{j=1}^N e_j sk_{2,j}^{(id^*)} \cdot sk_{1,j}^{(id)} \pmod{q_{K-1}}} \\
&= (\sigma_{\chi}^{F^*})^{r \cdot sk_0^{(id^*)-1} \cdot sk_0^{(id)}} = \left((H(c_{\chi} + \chi h_3(c_{\chi})) \cdot \text{Tag}^{(id^*, F^*)}) \right. \\
&\quad \left. \cdot W^{c_{\chi}} \right)^{sk_0^{(id^*)-1} \cdot sk_0^{(id)}} \\
&= \left((H(c_{\chi} + \chi h_3(c_{\chi})) \cdot \text{Tag}^{(id^*, F^*)}) \cdot W^{c_{\chi}} \right)^{r \cdot sk_0^{(id)}}.
\end{aligned}$$

Then, $\sigma_{\delta'_j}$ is calculated as follows:

$$\begin{aligned}
\sigma_{\delta'_j} &= \prod_{\delta'_j \in \{\delta'_j, \vartheta'_j\}} (\sigma_{\delta'_j,1} \sigma_{\delta'_j,2})^{r_0^{-1} \vartheta'_j} \\
&= \left(H(\text{Tag}^{(id,F)} \delta'_j h_3(c_{\chi})) - \text{Tag}^{(id^*, F^*)} \right. \\
&\quad \left. \chi h_3(c_{\chi})) \right)^{r \cdot \sum_{t=1}^K \xi_{0,t} (sk_{1,t}^{(id)} \pmod{q_{K-1}})} \\
&\quad \cdot \left(H(c_{\chi} + \chi h_3(c_{\chi})) \cdot \text{Tag}^{(id^*, F^*)} \cdot W^{c_{\chi}} \right)^{r \cdot sk_0^{(id)}} \\
&= \left(H(\text{Tag}^{(id,F)} \delta'_j h_3(c_{\chi})) - \text{Tag}^{(id^*, F^*)} \chi h_3(c_{\chi})) \right)^{r \cdot sk_0^{(id)}} \\
&\quad \cdot \left(H(c_{\chi} + \chi h_3(c_{\chi})) \cdot \text{Tag}^{(id^*, F^*)} \cdot W^{c_{\chi}} \right)^{r \cdot sk_0^{(id)}} \\
&= \left(H(c_{\chi} + \text{Tag}^{(id,F)} \delta'_j h_3(c_{\chi})) \cdot W^{c_{\delta'_j}} \right)^{r \cdot sk_0^{(id)}}.
\end{aligned}$$

Finally, we derive the correctness of the verification $e(\Theta, g) = e(H(\varphi + \Pi) \cdot W^{\varphi}, pk_0^{(id)})$, which is elaborated as follows:

$$\begin{aligned}
e(\Theta, g) &= e(\psi_0 \cdot \psi_1^{r^{-1}}, g) \\
&= e \left(\prod_{\delta_i \in \{\delta_i, \vartheta_i\} / \{\delta'_j, \vartheta'_j\}} \sigma_{\delta_i}^{\vartheta_i} \cdot \left(\prod_{\delta'_j \in \{\delta'_j, \vartheta'_j\}} \sigma_{\delta'_j}^{\vartheta'_j} \right)^{r^{-1}}, g \right) \\
&= e \left(\prod_{\delta_i \in \{\delta_i, \vartheta_i\} / \{\delta'_j, \vartheta'_j\}} \left(H(c_{\delta_i} + \text{Tag}^{(id,F)} \delta_i h_3(c_{\delta_j})) \right. \right. \\
&\quad \left. \cdot W^{c_{\delta_i}} \right)^{sk_0^{(id)} \vartheta_i} \cdot \left(\prod_{\delta'_j \in \{\delta'_j, \vartheta'_j\}} \left(H(c_{\delta'_j} + \text{Tag}^{(id,F)} \delta'_j \right. \right. \\
&\quad \left. \left. h_3(c_{\delta'_j})) \cdot W^{c_{\delta'_j}} \right)^{r \cdot sk_0^{(id)} \vartheta'_j} \right)^{r^{-1}}, g \right) \\
&= e \left(\prod_{j=1}^{\lambda} \left(H(c_{\delta_j} + \text{Tag}^{(id,F)} \delta_j h_3(c_{\delta_j})) \cdot W^{c_{\delta_j}} \right)^{\vartheta_j \beta_j}, g \right)
\end{aligned}$$

$$\begin{aligned}
&= e \left(H \left(\sum_{j=1}^{\lambda} \vartheta_j c_{\delta_j} + \text{Tag}^{(id,F)} \sum_{j=1}^{\lambda} \delta_j \vartheta_j h_3(c_{\delta_j}) \right) \right. \\
&\quad \left. \cdot W^{\sum_{j=1}^{\lambda} \vartheta_j c_{\delta_j}}, g^{\beta} \right) = e \left(H(\varphi + \Pi) \cdot W^{\varphi}, pk_0^{(id)} \right).
\end{aligned}$$

V. SECURITY ANALYSIS

Now we analyze the security of BA-FGDIA from three aspects: data confidentiality, OP protection, reliability of integrity auditing.

Theorem 1: BA-FGDIA achieves data confidentiality, provided the hardness assumption of the DL problem holds,

Proof: We first prove that BA-FGDIA achieves data confidentiality against CSP from the partial deduplication tags and final deduplication tags. Suppose $\mathcal{C}$ as a challenger, $\mathcal{A}_1$ as an adversary with the nonnegligible probability ϵ_k to attack partial deduplication tag $\text{tag}_i^{(id,F)} = pk_c^{sk_1^{(id)-1} + sk_1^{(id)} h_3(CC_i)}$ for id to get the ciphertext, where $\text{Enc}(h_5(m_i), m_i) = CC_i$ in $\text{tag}_i^{(id,F)}$.

- 1) *Setup:* $\mathcal{C}$ takes as input κ_0 , outputs $\{G_1, G_2, e, g, h_3\}$. Then $\mathcal{C}$ randomly chooses $sk_1^{(id)} = \alpha \in Z_p^*$, computes $\text{rpk}_1 = g^{\alpha^{-1}}$, and sends $\{pp, \text{rpk}_1\}$ to $\mathcal{A}_1$.
- 2) *Challenge:* $\mathcal{A}_1$ chooses two messages $\{m_0, m_1\}$, and submits $\{m_0, m_1\}$ to $\mathcal{C}$. Then $\mathcal{C}$ exploits a fair binary index coin to get b^* , computes $\text{tag}_{m_{b^*}}' = pk_c^{sk_1^{(id)-1} + sk_1^{(id)} h_3(m_{b^*})}$, returns it to $\mathcal{A}_1$.
- 3) *Guess:* $\mathcal{A}_1$ guesses b' of b^* , submits it to $\mathcal{C}$. If $b' = b^*$, $\mathcal{C}$ outputs 1. Otherwise, it outputs 0. In case of $b' = b^*$, such that

$$\Pr[b' = b^*] = \frac{1}{2} + \epsilon_k.$$

In this case, the advantage of $\mathcal{A}_1$ is ϵ_k . However, based on the hardness assumption of the DL problem, it is computationally infeasible for $\mathcal{A}_1$ to get $sk_1^{(id)-1} = \alpha^{-1}$ and $sk_1^{(id)} = \alpha$. Hence, we can get

$$\Pr[b' = b^*] = \frac{1}{2}.$$

Thus, we conclude $\mathcal{A}_1$ cannot obtain the ciphertext from the partial deduplication tags. Similarly, due to the hardness assumption of the DL problem, $\mathcal{A}_1$ cannot get the ciphertext in polynomial time from the tuple $\{pk_c, DB_{\text{Nodup}}^{(id,F)} = \{e(pk_c^{h_3(CC_i)}, g)\}\}$.

Then, we prove that BA-FGDIA achieves data confidentiality against CSP from launching BFA to obtain data block content. Since CSP stores the auxiliary encrypted information $\phi = \{g^{r_i^{(id,F)} h_3(m_i)^{-1}}, i \in [1, n]\}$ for F , if CSP holds $\mathcal{M} = \{\widehat{m}_1, \widehat{m}_2, \dots, \widehat{m}_t\}$, $t \gg n$, the adversary $\mathcal{A}_1$ maybe obtain the data block content through BFA by using ϕ . Actually, $\mathcal{A}_1$ needs to compute $g^{r_i^{(id,F)} h_3(\widehat{m}_i)^{-1}}$ and compare it with $g^{r_i^{(id,F)} h_3(m_i)^{-1}}$. $\mathcal{A}_1$ needs to master $r_i^{(id,F)}$ or $g^{r_i^{(id,F)}}$. Since CSP only knows $g^{r_i^{(id,F)} h_3(m_i)^{-1}}$, the complexity of computing $r_i^{(id,F)} h_3(m_i)^{-1}$ from $\{g, g^{r_i^{(id,F)} h_3(m_i)^{-1}}\}$ equals to solve the hardness assumption of the DL problem. It is computationally infeasible for $\mathcal{A}_1$ to deduce the data block content. In addition,

for a ciphertext c_i , the corresponding encryption key is $\text{key}_i = h_5(h_2(g^{r_i^{(id,F)}}, g^{\text{Tag}^{(id,F)}}, h_3(m_i)))$. If $\mathcal{A}_1$ possess $\text{Tag}^{(id,F)}$ and $g^{r_i^{(id,F)} h_3(m_i)^{-1}}$, it can compute $g^{\text{Tag}^{(id,F)}}$, but it cannot get $g^{r_i^{(id,F)}}$ from $\{g, g^{r_i^{(id,F)} h_3(m_i)^{-1}}\}$ based on the hardness assumption of the DL problem. Thus, $\mathcal{A}_1$ cannot launch BFA to compute an encryption key $\text{key}'_i = h_5(h_2(g^{r_i^{(id,F)}}, g^{\text{Tag}^{(id,F)}}, h_3(\widehat{m}_i)))$ to get c_i by encrypting $\widehat{m}_i$ and comparing it with c_i . ■

Theorem 2: BA-FGDIA achieves OP protection.

Proof: In *Duplication Detction*, it is difficult for an adversary $\mathcal{A}_2$ to exactly master which data blocks are owned by a user and which users own the duplicate data blocks. Suppose id and id^* own the same data blocks $\{m_1, m_2, \dots, m_i\}$. Since the final deduplication tags need to be generated jointly by a user and CSP, $\mathcal{A}_2$ can only try to utilize the partial duplication tags $\{pk_c^{sk_1^{(id)}-1+sk_1^{(id)} h_3(\text{Enc}(h_5(m_i), m_i))}\}$ for id and $\{pk_c^{sk_1^{(id^*)}-1+sk_1^{(id^*)} h_3(\text{Enc}(h_5(m_i), m_i))}\}$ for id^* to find out the relationship between id and id^* . Actually, without the private key ς of CSP, it is computationally infeasible for $\mathcal{A}_2$ in polynomial time to compute $pk_1^{(id)-\varsigma^{-1}}$ and $pk_1^{(id^*)-\varsigma^{-1}}$, such that cannot judge $pk_c^{sk_1^{(id)}-1+sk_1^{(id)} h_3(\text{Enc}(h_5(m_i), m_i))} = pk_c^{sk_1^{(id^*)}-1+sk_1^{(id^*)} h_3(\text{Enc}(h_5(m_i), m_i))}$. Thus, the partial deduplication tags will not reveal the OP of id and id^* .

Besides, it is obvious that $\mathcal{A}_2$ cannot obtain the OP of other users who possess the backups of duplicated data blocks owned by id from the detection result R . $\mathcal{A}_2$ may obtain the OP of other users id^* in the PoW protocol, in which CSP sends a partial encryption key set $\mathcal{B} = \{g^{h_5(\text{Tag}^{(id_i, F_i)}), g^{r_{i_i}^{(id_i, F_i)} h_3(m_i)^{-1}}\}$ to id for duplicated data blocks. In $\mathcal{B}$, $\mathcal{A}_2$ regards $g^{h_5(\text{Tag}^{(id_i, F_i)})}$ as the attack target and tries to break the OP of id_i . However, due to the hardness assumption of the DL problem, it is also computationally infeasible for $\mathcal{A}_2$ to get $\text{Tag}^{(id_i, F_i)}$ in polynomial time.

Furthermore, when CSP generates the integrity auditing proof information, TP resigns and generates an authentication tag. When TP receives the signature request $\{id \rightarrow id^*, \text{Token}, q_{K-1}, H(\text{Tag}^{(id,F)} \delta'_j h_3(c_\chi^{F*}) - \text{Tag}^{(id^*, F^*)} \chi h_3(c_\chi^{F*}))^{r_0}, \sigma_\chi^{(F^*)^{r_0}}\}$, TP tries to get the OP $\text{Tag}^{(id,F)}$, $\text{Tag}^{(id^*, F^*)}$ and c_χ^{F*} from $H(\text{Tag}^{(id,F)} \delta'_j h_3(c_\chi^{F*}) - \text{Tag}^{(id^*, F^*)} \chi h_3(c_\chi^{F*}))^{r_0}$ and $\sigma_\chi^{(F^*)^{r_0}}$. Actually, TP does not master the random factor r_0 and the signed private key $sk_0^{id^*}$, and thus based on the hardness assumption of the DL and CDH problems, it is also computationally infeasible for $\mathcal{A}_2$ to get $\text{Tag}^{(id,F)}$, $\text{Tag}^{(id^*, F^*)}$, and c_χ^{F*} from the signature request in polynomial time, so TP cannot determine which users own the duplicate data block c_χ^{F*} .

For the integrity auditing logs, suppose there exists a public log set $\{\log^{(id^*, F^*)} = \{h_3(\text{Tag}^{(id^*, F^*)}), VI, \text{Proof}, \text{Result}\}\}$ with the same timestamp Time_2 on the blockchain. if $\mathcal{A}_2$ obtains two integrity auditing logs $\log^{(id,F)} = \{h_3(\text{Tag}^{(id,F)}), VI, \text{Proof}, \text{Result}\}$ and $\log^{(id^*, F^*)} = \{h_3(\text{Tag}^{(id^*, F^*)}), VI^*, \text{Proof}^*, \text{Result}^*\}$ corresponded to id and id^* , respectively. Essentially, it is computationally infeasible for $\mathcal{A}_2$ to know whether there exist the same data blocks

from two integrity auditing logs, since id and id^* have different integrity auditing tokens $\text{Token} = r||\text{Time}_2$ and $\text{Token}^* = r^*||\text{Time}_2$ in which $r \neq r^*$, and use different signing private keys, such that there exist $\psi_0 \neq \psi_0^*$ and $\psi_1 \neq \psi_1^*$. Even in the worst circumstance, where the challenging ciphertexts of id and id^* are identical, the aggregated ciphertext block $\varphi \neq \varphi^*$ in *Proof* and *Proof*^{*}. Since in $\text{Chal}^{(id,F)} = \{h_3(\text{Tag}^{(id,F)}), \text{Token}, \theta_1, \theta_2, \lambda\}$ and $\text{Chal}^{(id^*, F^*)} = \{h_3(\text{Tag}^{(id^*, F^*)}), \text{Token}^*, \theta_1^*, \theta_2^*, \lambda\}$ on the blockchain, there exist random source seeds $\theta_1 \neq \theta_1^*$ and $\theta_2 \neq \theta_2^*$ due to $\text{Token} \neq \text{Token}^*$ and $r^{-1} \neq r^{*-1}$, and the corresponding challenge coefficients are different, it is also computationally infeasible for $\mathcal{A}_2$ to distinguish whether the corresponding $\text{Tag}^{(id,F)}$ and $\text{Tag}^{(id^*, F^*)}$ have duplicated data blocks within polynomial time, making it more difficult to know which two users have duplicated data blocks due to no identity information of users in the public information on blockchain. Thus, BA-FGDIA can protect the OP of users. ■

Theorem 3: BA-FGDIA achieves storage correctness guarantee against integrity auditing proof forgery, provided the hardness assumptions of the CDH and DL problems hold.

Proof: We prove BA-FGDIA achieves storage correctness guarantee according to the following two games.

Game₀: The challenger $\mathcal{C}$ generates the public parameters set and the signature key pairs $\{sk_0^{(id_i)}, pk_0^{(id_i)}\}$ of all users. Then $\mathcal{C}$ sends all public keys set $\{pk_0^{(id_i)}\}$ to an adversary $\mathcal{A}_3$. $\mathcal{A}_3$ can query the corresponding signatures of ciphertext blocks. $\mathcal{C}$ submits the challenge information $\text{Chal} = \{g^{\text{Tag}^{(id_i, F)}}, \text{Token}, \theta_1, \theta_2, \lambda\}$ to $\mathcal{A}_3$, and gets the integrity auditing proof information $\text{Proof} = \{\psi, \varphi\}$. Meanwhile, $\mathcal{C}$ maintains a list of integrity auditing proof about the aggregate signature. If the aggregate signature $\psi^* = \prod_{i=1}^{\lambda} \sigma_{\delta_i}^{\vartheta_i}$ is not in the list, but it passes the integrity auditing, $\mathcal{C}$ aborts the *Game₀*. If *Proof* is valid, $\mathcal{A}_3$ wins *Game₀*.

Analysis: We assume that there exists a valid integrity auditing proof information $\text{Proof} = \{\psi, \varphi\}$ for the $\text{Tag}^{(id^*, F)}$, such that

$$e(\psi, g) = e\left(\prod_{i=1}^{\lambda} \sigma_{\delta_i}^{\vartheta_i}, g\right) = e\left(\prod_{i=1}^{\lambda} H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)})^{\vartheta_i} \cdot W^{\varphi}, pk_0^{id^*}\right). \quad (2)$$

If $\mathcal{A}_4$ can forge an aggregate signature ψ^* , $\psi^* \neq \psi$. Especially, the forged proof information $\text{Proof}^* = \{\psi^*, \varphi^*\}$ can satisfy the verification equation. Thus

$$e(\psi^*, g) = e\left(\prod_{i=1}^{\lambda} \sigma_{\delta_i}^{*\vartheta_i}, g\right) = e\left(\prod_{i=1}^{\lambda} H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)})^{\vartheta_i} \cdot W^{\varphi^*}, pk_0^{id^*}\right). \quad (3)$$

We could know there has at least a ciphertext block $c_{\delta_i} \neq c_{\delta_i}^*$ such that $\Delta\varphi = \varphi - \varphi^* \neq 0$. We construct a simulator SIM_1 to solve the hardness assumption of the CDH problem given the tuple $\{g, pk_0^{(id^*, F)} = g^\beta, Q\}$. SIM_1 randomly chooses $a, b \in \mathbb{Z}_p^*$ and sets $W = g^a Q^b$. Additionally, SIM_1 implements a hash query, it randomly chooses $r_i \in \mathbb{Z}_p$ and utilizes random oracle to get $H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)}) = [g^{r_i} / (g^a Q^b)^{c_{\delta_i}}]$.

Thus, SIM_1 is able to compute σ_{δ_i} without the signature private key $sk_0^{id^*} = \beta$

$$\begin{aligned}\sigma_{\delta_i} &= \left(H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)}) \cdot W^{c_{\delta_i}} \right)^{sk_0^{id^*}} \\ &= \left(\frac{g^{r_i}}{(g^a Q^b)^{c_{\delta_i}}} \cdot g^{ac_{\delta_i}} Q^{bc_{\delta_i}} \right)^{sk_0^{id^*}} = (g^{r_i})^{sk_0^{id^*}} = pk_0^{(id^*, F) r_i}.\end{aligned}$$

Then, we get

$$\begin{aligned}e\left(\frac{\psi}{\psi^*}, g\right) &= e\left(H(\varphi - \varphi^*) \cdot w^{\varphi - \varphi^*}, pk_0^{id^*}\right) \\ &= e\left(H(\Delta\varphi) \cdot (g^a Q^b)^{\Delta\varphi}, g^\beta\right).\end{aligned}$$

Further, we can obtain

$$e\left(\frac{\psi}{\psi^*} \cdot H(\Delta\varphi)^{-\beta} \cdot g^{-a\Delta\varphi\beta}, g\right) = e(Q^\beta, g)^{\Delta\varphi\beta}.$$

If $\Delta\varphi \neq 0$, SIM_1 could compute $Q^\beta = ([\psi/\psi^*] \cdot H(\Delta\varphi)^{-\beta} \cdot g^{-a\beta\Delta\varphi})^{(1/b\Delta\varphi)}$. The probability of $\Delta\varphi = 0$ is $(1/p)$. Thus, the nonnegligible probability that $\mathcal{A}_3$ solves the hardness assumption of the CDH problem is at least $1 - [1/p]$, which leads to the contradiction.

Game₁: It is similar to *Game₀*, except that there exists the challenger $\mathcal{C}$ maintaining a list of integrity auditing proof about the aggregated data blocks. If the aggregated data block $\varphi^* = \sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}^*$ is not in the list but could pass the integrity auditing, $\mathcal{C}$ aborts *Game₁*. If Proof is valid, the adversary $\mathcal{A}_4$ wins the *Game₁*.

Analysis: We assume there exists a valid integrity proof information $\text{Proof} = \{\psi, \varphi\}$ for $\text{Tag}^{(id^*, F)}$, in which $\varphi = \sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}$, such that

$$\begin{aligned}e(\psi, g) &= e\left(\prod_{i=1}^{\lambda} \sigma_{\delta_i}^{\vartheta_i}, g\right) = e\left(\prod_{i=1}^{\lambda} H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)})^{\vartheta_i} \cdot W^{\varphi}, pk_0^{id^*}\right).\end{aligned}\quad (4)$$

If $\mathcal{A}_4$ can forge an aggregated data block $\varphi^* = \sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}^*$, in which $\Delta\varphi = \varphi - \varphi^* \neq 0$, such that

$$\begin{aligned}e(\psi, g) &= e\left(\prod_{i=1}^{\lambda} \sigma_{\delta_i}^{\vartheta_i}, g\right) = e\left(\prod_{i=1}^{\lambda} H(c_{\delta_i}^* + \delta_i h_3(c_{\delta_i}^*) \text{Tag}^{(id^*, F)})^{\vartheta_i} \cdot W^{\varphi^*}, pk_0^{id^*}\right).\end{aligned}\quad (5)$$

Thus, $e(H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)})^{\vartheta_i} \cdot W^{\varphi}, pk_0^{id^*}) = e(H(c_{\delta_i} + \delta_i h_3(c_{\delta_i}) \text{Tag}^{(id^*, F)})^{\vartheta_i} \cdot W^{\varphi^*}, pk_0^{id^*})$. Furthermore, we can obtain $H(\varphi)W^{\sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}} = H(\varphi^*)W^{\sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}^*}$.

We construct a simulator SIM_2 to solve the hardness assumption of the DL problem given the tuple $\{g, Q = g^\alpha\}$. SIM_2 randomly chooses $a, b \in \mathbb{Z}_p$ and sets $W = g^a Q^b$. Then, $H(\Delta\varphi)W^{\sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i}} = 1_{G_1}$ such that $H(\varphi)W^{\sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}} = H(\varphi^*)W^{\sum_{i=1}^{\lambda} \vartheta_i c_{\delta_i}^*}$. SIM_2 randomly chooses $c \in \mathbb{Z}_p$ satisfying $H(\Delta\varphi) = g^a Q^c$. Thus, we get that

$$\begin{aligned}H(\Delta\varphi)W^{\sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i}} &= 1_{G_1} \\ \Rightarrow g^a Q^c (g^a Q^b)^{\sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i}} &= 1_{G_1} \\ \Rightarrow Q^{c+b \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i}} &= g^{-(a+a \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i})} \\ \Rightarrow Q &= g^\alpha = g^{\frac{-(a+a \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i})}{(c+b \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i})}}.\end{aligned}$$

We conclude $\mathcal{A}_4$ can compute $\alpha = [-(a + a \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i}) / (c + b \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i})]$, $c + b \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i} \neq 0$. On account of $\Delta\varphi \neq 0$ and $b, c \in \mathbb{Z}_p$, the probability of $c + b \sum_{i=1}^{\lambda} \vartheta_i \Delta c_{\delta_i} = 0$ is $(1/p - 1)$. Therefore, $\mathcal{A}_4$ can solve the hardness assumption of the DL problem with the probability $1 - (1/p - 1)$, which leads to the contradiction.

Accordingly, BA-FGDIA achieves storage correctness guarantee against integrity auditing proof forgery. ■

VI. PERFORMANCE EVALUATION

We conduct the comprehensive performance comparison with existing schemes, including BEDA [23], BASD [22], ALDAD [21], ETDA [20], and BDIA [19]. The whole experimental simulation utilizes Intel i5-12500H processor running on 2.50-GHz Win11 operating system with 16 GB of RAM. All algorithms are achieved using Java language and its JPBC library 2.0.0, where *Type-A* prime degree curve with field size of 512-bits is selected for elliptic curve, and $|p| = 160$ -bits. We respectively denote T_e , T_{Add} , T_{mul} , T_{exp} , T_{Mul} , T_H , T_{ec} , T_{pr} , and T_{hash} as bilinear pairing operation, additive operation in G_1 , normal multiplicative operation, exponential operation in G_1 , multiplicative operation in G_1 , hash-to-point function operation, the symmetric encryption operation, pseudo random number generation operation, and normal hash operation, where all experimental results are averaged over 100 trials.

A. Functionality Comparison

We list the functionality comparison of these schemes with deduplication and integrity auditing in Table I, where R-DFA stands for the resistance to DFA, and dup-tag denotes the authentication tag deduplication. The functionality comparison demonstrates that the scheme in [23] cannot protect data confidentiality. The schemes in [19] and [20] and BA-FGDIA achieve OP protection, while other schemes do not protect OP. For data deduplication, except the scheme in [22], the authentication tags deduplication and ciphertext deduplication are implemented. In particular, all previous schemes are devised based on file level deduplication, while BA-FGDIA is more fine-grained deduplication considering deduplication in data blocks, which has significant advantage. More attractively, although BA-FGDIA implements some same functionality in data deduplication as schemes [19], [20], we also improve the integrity auditing functionality compared to other schemes, such as maintaining lightweight verification advantages, deploying smart contracts to verify data integrity instead of traditional TPA, and implementing log checking in integrity auditing.

TABLE I
COMPARISON OF FUNCTIONALITY

Schemes	Confidentiality	R-DFA	Data Deduplication			Data Integrity Auditing			
			Dup-tag	OP	Fine-Grained	Batch	Log Checking	Lightweight	No-TPA
BEDA	NO	NO	YES	NO	NO	NO	YES	NO	NO
BASD	YES	NO	NO	NO	NO	YES	NO	NO	NO
ETDA	YES	NO	YES	YES	NO	YES	NO	NO	YES
ALDAD	YES	NO	YES	NO	NO	YES	NO	NO	NO
BDIA	YES	YES	YES	YES	NO	NO	YES	NO	NO
BA-FGDIA	YES	YES	YES	YES	YES	YES	YES	YES	YES

TABLE II
COMPARISON OF STORAGE ON BLOCKCHAIN

Schemes	Setup	Data Uploading	Integrity Auditing		Total Costs (bits)
			Proof Generation	Auditing Log	
BEDA	-	$N Z_P $	$4 Z_P + G_1 $	$4 Z_P + G_1 + 1$	$2 G_1 + (8 + N) Z_P + 1$
ETDA	-	-	$(N + 1) G_1 + 2N Z_P $	-	$(N + 1) G_1 + 2N Z_P $
BDIA	-	$t(n + 1) G_1 $	$5 G_1 + (4 + \lambda) Z_P $	$ G_1 + 1$	$(tn + t + 6) G_1 + (4 + \lambda) Z_P + 1$
BA-FGDIA	$t G_1 $	$(n + 1) Z_P $	$6 Z_P + G_1 $	$7 Z_P + 1$	$(1 + t) G_1 + (n + 14) Z_P + 1$

TABLE III
COMPARISON OF STORAGE COSTS AND COMPUTATIONAL COSTS IN DATA UPLOADING

Schemes	Storage Costs (bits)		Data Uploading	
	Initial uploader	Subsequent uploader	Initial uploading	Subsequent uploading
ETDA		$5 Z_P + G_1 + nl$	$(2n + 3)T_{hash} + (nd + n + 2)T_{exp}$ $+ nT_H + n(d + 1)T_{Mul} + nT_{ec}$	$nT_{hash} + 2T_H + nT_{ec}$
ALDAD		$ Z_P + 2 G_1 + nl$	$(2n + 2)T_{hash} + (nd + n + 2)T_{exp}$ $+ nT_H + (nd + n)T_{Mul} + nT_{ec}$	$\lambda T_{hash} + T_H + \lambda T_{mul} + T_{exp} + \lambda T_{ec}$
BDIA		$2 Z_P + G_1 + nl$	$(n + 1)T_{pr} + (nt + 3n + t + 3)T_{exp}$ $+ (nt + t)T_e + (3n + 2)T_H$ $+ (n + 1)T_{Mul} + nT_{ec}$	$(n + 1 + \lambda)T_{pr} + (nt + n + t$ $+ \lambda + 1)T_{exp} + (nt + t)T_e + (2n + 2 +$ $\lambda)T_H + \lambda T_{Mul} + \lambda T_{ec} + \lambda T_{mul}$
BA-FGDIA	$ Z_P + nl$	$3 Z_P + (n - c_{in})l$	$4nT_{hash} + 4nT_{exp}$ $+ nT_H + nT_{Mul} + nT_{ec}$	$(3c_{ex} + n)T_{hash} + c_{ex}T_{exp} + nT_{ec}$ $+ (n - c_{ex} - c_{in})(3T_{hash} + 4T_{exp}$ $+ T_H + T_{Mul})$

B. On-Chain Overhead

The blockchain platform is the hyperledger fabric v2.4.1. We deploy four peer nodes including endorers and committers, and two orderers, the MaxMessageCount is set to be 500 and AbsoluteMaxBytes are 15M, while deploying smart contracts written by Java language and choosing the Raft consensus algorithm. As these schemes in [19], [20], and [23], they all exploit the public blockchain that is required to pay gas fees. Considering the actual application scenario, we deploy the fabric blockchain in BA-FGDIA without requiring gas fees. As such, we compare the blockchain storage costs of each scheme

and evaluate the time costs of transactions to be packaged on the blockchain in BA-FGDIA.

In Table II, we list the comparison of the storage costs on the blockchain in different processes. In BA-FGDIA, the subsecret verification parameters are placed on the blockchain with the storage costs as $t|G_1|$. A user puts the integrity verification parameters on blockchain with the storage costs as $(n + 1)|Z_P|$. Upon receiving the integrity auditing request, the challenge information and proof information are generated and put on the blockchain with storage costs as $6|Z_P| + |G_1|$. Finally, the integrity auditing log is also stored on the

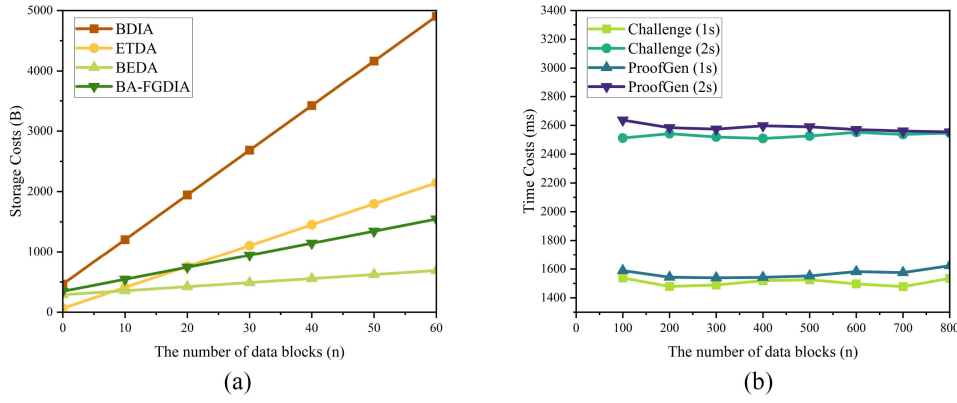


Fig. 2. Comparison of blockchain costs. (a) Total storage costs. (b) Time costs of challenge generation and proof generation on blockchain.

blockchain which consumes the storage costs as $7|Z_p| + 1$. Thus, the total storage costs of blockchain are $(1+t)|G_1| + (n+14)|Z_p| + 1$. The Fig. 2(a) shows the comparison of storage costs on blockchain in which let $N = n$. We get the storage costs of BA-FGDIA and the other schemes increase linearly with the number of data blocks, and the storage costs of BA-FGDIA are lower than the scheme in [19]. The storage costs of BA-FGDIA are slightly higher than the scheme in [23], to avoid the forged signature of CSP through TP to pass integrity auditing and the malicious deduplication of CSP, it is inevitable to place auditing logs containing auxiliary verification parameters, signature token and verification information on the blockchain for the reliability of integrity verification, but the scheme in [22] does not implement log checking.

Then, we analyze the time costs of the transactions (Challenge, ProofGen) generated in integrity auditing to be packaged on the blockchain when the batch timeout is set to be 1 and 2 s. The time costs on blockchain are depicted in Fig. 2(b), the experimental simulation results demonstrate that as the number of the challenged data blocks increases, the time costs including the running time of smart contracts SC_1 or SC_2 , the transaction batched to blockchain and the irresistible network fluctuations keep in a constant range facing different batch timeout. Therefore, BA-FGDIA is suitable to be deployed in blockchain-assisted cloud storage systems.

C. Off-Chain Overhead

We first present the comparison of storage and computational costs within uploading a data file for the user-side in schemes [19], [20], [21] in Table III. In BA-FGDIA, when a user outsources the data file F to CSP, the user determines whether data blocks already exist on the cloud. In initial uploading, the user first generates file tag and n encryption keys with the computational costs as $3nT_{\text{hash}} + 2nT_{\text{exp}}$, then encrypts n data blocks with the computational costs as nT_{ec} . After obtaining the ciphertext, the computational costs of generating authentication tags are $n(T_H + 2T_{\text{exp}} + T_{\text{Mul}})$. Thus, the total computational costs of initial uploading are $4nT_{\text{hash}} + 4nT_{\text{exp}} + nT_H + nT_{\text{Mul}} + nT_{\text{ec}}$, and corresponding storage costs are $|Z_p| + nl + 2$, in which the user only stores the deduplication credential $cred$ with the size as $|Z_p| + 2$ and n

encryption keys with the size as nl . In subsequent uploading, a user gets the number of internal duplication c_{in} and external duplication c_{ex} from deduplication credential $cred$. Then, the user calculates the encryption keys of the unduplicated blocks with the computational costs as $(n - c_{in} - c_{ex})(3T_{\text{hash}} + 2T_{\text{exp}} + T_{\text{ec}})$, the authentication tags with the computational costs as $(n - c_{in} - c_{ex})(T_H + T_{\text{Mul}} + T_{\text{exp}})$, and the ciphertext duplicate blocks with the computational costs as $3c_{ex}T_{\text{hash}} + c_{ex}T_{\text{exp}} + c_{ex}T_{\text{ec}}$. The total computational costs of subsequent uploading are $(n - c_{in} - c_{ex})(3T_{\text{hash}} + T_H + T_{\text{Mul}} + 4T_{\text{exp}}) + 3c_{ex}T_{\text{hash}} + c_{ex}T_{\text{exp}} + nT_{\text{ec}}$, and corresponding storage costs are $3|Z_p| + (n - c_{in})l + 2$.

We evaluate the storage and computational costs on user side in initial uploader and subsequent uploader. For better comparison, we use quantitative analysis on some variables in schemes [19], [20] and [21]. We set $t = 10$, $d = 16$, $|l| = 160$ -bits. In BA-FGDIA, we set the internal duplication rate to be 15%. Fig. 3 shows a specific comparison of user-side computational costs. Fig. 3(a) and (c) show the storage costs and computational costs change as the number of data blocks increases. When a user as the initial uploader outsources the data file F to CSP, we get that BA-FGDIA has lower storage and computational costs than the schemes in [19], [20], and [21]. In subsequent uploading process, Fig. 3(b) shows the storage costs of BA-FGDIA are still lower than other schemes, and will be lower than the initial uploading phase in BA-FGDIA, since with the number of internal duplicate blocks increases, the storage costs will decrease accordingly, showing negative growth. However, Fig. 3(d) shows that although the computational costs of BA-FGDIA are significantly lower than that of scheme [19], it is higher than that of schemes [20] and [21]. This is because in order to achieve fine-grained deduplication, OP protection and key security of low-entropy data in the PoW protocol, the target of BA-FGDIA is the data blocks, the computational costs will be affected by the data blocks and external duplication rate. For example, we need to calculate the key to obtain the duplicate ciphertext without disclosing the identity of other users. However, the object of [20] and [21] is a completely duplicate file, it does not carry out more fine-grained calculations and their encryption keys are directly hashed, there exists brute force attack problem between them.

TABLE IV
COMPARISON OF COMPUTATIONAL COSTS IN INTEGRITY AUDITING

Schemes	User	Proof Generation	Integrity Auditing	
			Signal Auditing	Batch Auditing
BASD	$\lambda T_{pr} + \lambda T_H$	$\lambda(T_{exp} + T_{Mul} + T_{mul} + T_{add}) + T_e + T_{mul}$	$2T_e + (\lambda + 3)T_{exp} + \lambda T_H + (\lambda + 1)T_{Mul}$	$(k + 1)T_e + (\lambda + k + 2)T_{exp} + \lambda T_H + (\lambda + k + 1)T_{Mul}$
ETDA	-	$n(T_{exp} + T_{Mul}) + dn$ $(T_{mul} + T_{add}) + T_{exp} + T_{mul}$	$2T_e + (n + 2)T_{exp} + nT_H + (n + 1)T_{Mul}$	$2T_e + (kn + 2)T_{exp} + knT_H + (kn + 1)T_{Mul}$
ALDAD	-	$\lambda(T_{exp} + T_{Mul}) + d\lambda(T_{mul} + T_{add})$ $(\lambda_1 + 3\lambda_2 + t\lambda_2 + \lambda_3t + 2\lambda_3)T_{exp}$	$2T_e + (\lambda + d)T_{exp} + \lambda T_H + (\lambda + d)T_{Mul}$	$2T_e + (\lambda + d)T_{exp} + \lambda T_H + (\lambda + d + 2k)T_{Mul}$
BA-FGDIA	T_{ec}	$+(\lambda_2 + \lambda_3)T_H + (\lambda + t\lambda_2 + t\lambda_3)T_{Mul} + \lambda T_{add} + \lambda T_{mul}$	$2T_e + 2T_{Mul} + 2T_{exp} + T_H$	$2T_e + 2T_{Mul} + 2T_{exp} + T_H$

Note: $\lambda = \lambda_1 + \lambda_2 + \lambda_3$, where λ_1 , λ_2 and λ_3 are respectively the challenge blocks number of unduplication, internal duplication and external duplication.

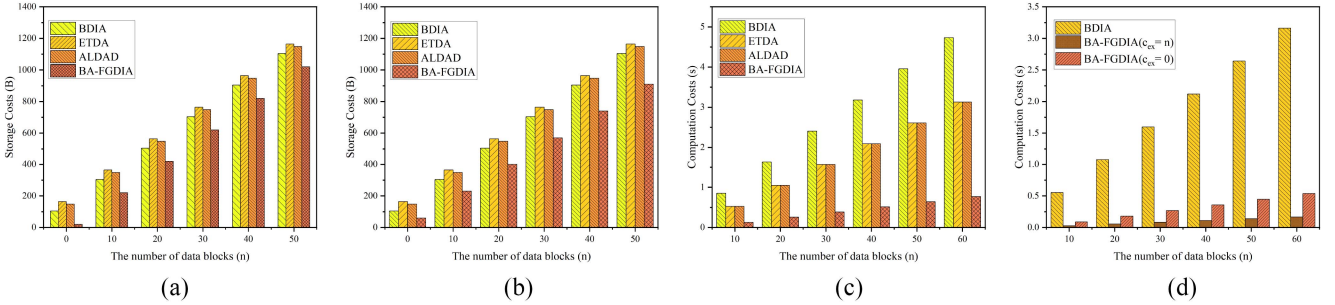


Fig. 3. Comparison of costs on user side. (a) Storage costs of initial uploader. (b) Storage costs of subsequent uploader. (c) Computational costs in initial uploading. (d) Computational costs in subsequent uploading.

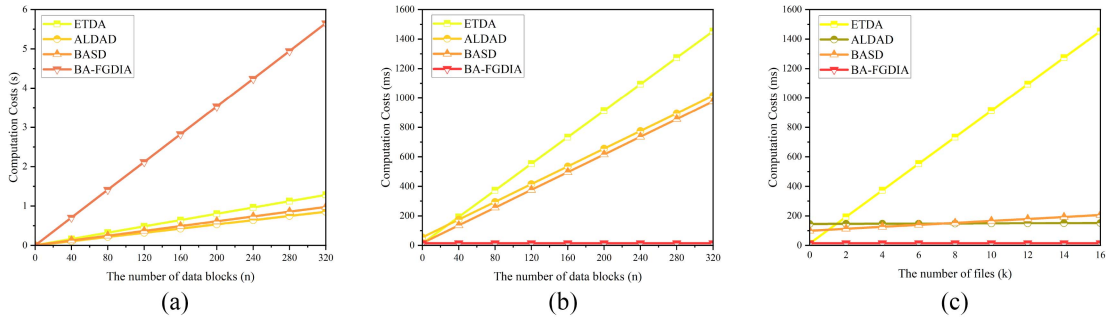


Fig. 4. Comparison in integrity auditing. (a) Computational costs of proof generation. (b) Computational costs of signal integrity auditing. (c) Computational costs of batch integrity auditing.

In Table IV, we list the computational costs in integrity auditing. In BA-FGDIA, a user sends an auditing request with calculating a resignature token where the computational costs are T_{ec} . Then CSP calculates the proof information of the number of deduplication data blocks and the integrity auditing proof information with the computational costs as $(\lambda_1 + 3\lambda_2 + t\lambda_2 + \lambda_3t + 2\lambda_3)T_{exp} + (\lambda_2 + \lambda_3)T_H + (\lambda_1 + \lambda_2 + \lambda_3 + t\lambda_2 + t\lambda_3)T_{Mul} + \lambda T_{add} + \lambda T_{mul}$, where CSP needs to interact with TP to get the resignatures, and finally the computational costs of integrity auditing proof are $2T_e + 2T_{Mul} + 2T_{exp} + T_H$. In Table IV, we also give the computational costs of each entity in

batch auditing. Although the user needs to calculate additional signature token when performing integrity auditing compared with schemes [19] and [20], this token not only restricts the availability and timeliness of auditing and resignature, but also prevents the malicious behaviors of CSP, such as proof information reuse and malicious resignature, and it is acceptable and only accounts for the very small part of the total costs.

The comparison is given in Fig. 4, where we set the number of random challenge blocks to be $\lambda = (2/3)n$ and $\lambda_1 = \lambda_2 = \lambda_3 = [1/3]\lambda$. Fig. 4(a) shows the computational costs

of integrity auditing proof generation in CSP, where BA-FGDIA has a slightly higher computational costs, because in order to protect the security of signature key, we deploy the trust proxies to assist CSP to compute the authentication tags corresponded to challenge blocks, thus the additional computational costs of CSP are acceptable under the strong computation power. Meanwhile, it is also reflected in Fig. 3(c) that BA-FGDIA has achieved lower computational costs on the user-side than other schemes. Fig. 4(b) and (c) show that the computational costs of BA-FGDIA are lower than other schemes and do not increase with the number of data blocks or files in signal auditing and batch auditing, BA-FGDIA has more lightweight advantages in integrity auditing.

VII. CONCLUSION

In this article, we have devised the BA-FGDIA scheme for large-scale data in cloud storage, which exploits smart contracts to replace the traditional TPA to avoid malicious behaviors and supervise malicious conspiracy behaviors with the cloud server. The proposed scheme achieves data fine-grained deduplication and guarantees the recovery of the deduplication data. We have proved the scheme guarantees data confidentiality, ensures OP protection and reliability of integrity auditing. We have conducted the performance evaluation, conducted the simulation of blockchain. The results demonstrate that our scheme has lower computational costs on user side, lightweight integrity auditing costs, and is also suitable for deployment in resource-constrained devices.

REFERENCES

- [1] J. Singh, G. Singh, and A. Verma, "The anatomy of big data: Concepts, principles and challenges," in *Proc. 8th Int. Conf. Adv. Comput. Commun. Syst. (ICACCS)*, 2022, pp. 986–990.
- [2] Y. Miao, R. H. Deng, X. Liu, K.-K. R. Choo, H. Wu, and H. Li, "Multi-authority attribute-based keyword search over encrypted cloud data," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 4, pp. 1667–1680, Jul./Aug. 2021.
- [3] F. Luo, H. Wang, W. Susilo, X. Yan, and X. Zheng, "Public trace-and-revoke proxy re-encryption for secure data sharing in clouds," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 2919–2934, 2024.
- [4] X. Li et al., "An identity-based data integrity auditing scheme for cloud-based maritime transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2556–2567, Feb. 2023.
- [5] D. Reinsel, J. Gantz, and J. Rydning, "Data age 2025: The digitization of the world from edge to core," IDC, Needham, MA, USA, White Paper, 2018.
- [6] G. Ateniese et al., "Provable data possession at untrusted stores," in *Proc. 14th ACM Conf. Comput. Commun. Secur.*, 2007, pp. 598–609.
- [7] A. Yang, J. Xu, J. Weng, J. Zhou, and D. S. Wong, "Lightweight and privacy-preserving delegatable proofs of storage with data dynamics in cloud storage," *IEEE Trans. Cloud Comput.*, vol. 9, no. 1, pp. 212–225, Jan.–Mar. 2021.
- [8] S. Li, C. Xu, Y. Zhang, Y. Du, and K. Chen, "Blockchain-based transparent integrity auditing and encrypted deduplication for cloud storage," *IEEE Trans. Services Comput.*, vol. 16, no. 1, pp. 134–146, Jan./Feb. 2023.
- [9] T. Li, J. Chu, and L. Hu, "CIA: A collaborative integrity auditing scheme for cloud data with multi-replica on multi-cloud storage providers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 1, pp. 154–162, Jan. 2023.
- [10] M. Wang, J. Yu, W. Shen, and R. Hao, "Privacy-preserving time-based auditing for secure cloud storage," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7866–7878, 2024.
- [11] F. Wang, L. Xu, J. Li, and K.-K. R. Choo, "Lightweight public/private auditing scheme for resource-constrained end devices in cloud storage," *IEEE Trans. Cloud Comput.*, vol. 10, no. 4, pp. 2704–2716, Oct.–Dec. 2022.
- [12] X. Li, S. Liu, R. Lu, M. K. Khan, K. Gu, and X. Zhang, "An efficient privacy-preserving public auditing protocol for cloud-based medical storage system," *IEEE J. Biomed. Health Inform.*, vol. 26, no. 5, pp. 2020–2031, May 2022.
- [13] J. Zhao, H. Huang, D. He, X. Zhang, Y. Zhang, and K.-K. R. Choo, "IB-IADR: Enabling identity-based integrity auditing and data recovery with fault Localization for multicloud storage," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 27214–27231, Aug. 2024.
- [14] J. R. Douceur, A. Adya, W. J. Bolosky, P. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. 22nd Int. Conf. Distrib. Comput. Syst.*, 2003, pp. 617–624.
- [15] M. Bellare, S. Keelveedhi, and T. Ristenpart, *Message-Locked Encryption and Secure Deduplication*, Int. Assoc. Cryptologic Res. Co., Bellevue, WA, USA, Jan. 2012.
- [16] Y. Shin, D. Koo, J. Yun, and J. Hur, "Decentralized server-aided encryption for secure deduplication in cloud storage," *IEEE Trans. Services Comput.*, vol. 13, no. 6, pp. 1021–1033, Nov./Dec. 2020.
- [17] D. Quick and K.-K. R. Choo, "Google drive: Forensic analysis of data remnants," *J. Netw. Comput. Appl.*, vol. 40, pp. 179–193, Apr. 2014.
- [18] Z. Hua, Y. Yao, M. Song, Y. Zheng, Y. Zhang, and C. Wang, "Blockchain-assisted secure deduplication for large-scale cloud storage service," *IEEE Trans. Services Comput.*, vol. 17, no. 3, pp. 821–835, May/Jun. 2024.
- [19] M. Song, Z. Hua, Y. Zheng, H. Huang, and X. Jia, "Blockchain-based deduplication and integrity auditing over encrypted cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 6, pp. 4928–4945, Nov./Dec. 2023.
- [20] M. Song, Z. Hua, Y. Zheng, T. Xiang, and X. Jia, "Enabling transparent deduplication and auditing for encrypted data in cloud," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3545–3561, Jul./Aug. 2024.
- [21] X. Gao et al., "Achieving low-entropy secure cloud data auditing with file and authenticator deduplication," *Inf. Sci.*, vol. 546, pp. 177–191, Feb. 2021.
- [22] H. Yuan, X. Chen, J. Wang, J. Yuan, H. Yan, and W. Susilo, "Blockchain-based public auditing and secure deduplication with fair arbitration," *Inf. Sci.*, vol. 541, pp. 409–425, Dec. 2020.
- [23] Y. Xu, C. Zhang, G. Wang, Z. Qin, and Q. Zeng, "A blockchain-enabled deduplicatable data auditing mechanism for network storage services," *IEEE Trans. Emerg. Topics Comput.*, vol. 9, no. 3, pp. 1421–1432, Jul.–Sep. 2021.
- [24] A. Juels and B. S. Kaliski Jr., "PORs: Proofs of retrievability for large files," Cryptol. ePrint Arch., IACR, Bellevue, WA, USA, Rep.2007/243, 2007.
- [25] H. Shacham and B. Waters, *Compact Proofs of Retrievability*, IACR, Bellevue, WA, USA, 2008.
- [26] S. Li et al., "Backdoor-resistant public data integrity verification scheme based on smart contracts," *IEEE Internet Things J.*, vol. 10, no. 16, pp. 14269–14284, Aug. 2023.
- [27] H. Zhang, X. Zhang, Y. Zhang, X. Wang, and Q. Liu, "Blockchain-based proxy-oriented data integrity checking mechanism in cloud-assisted intelligent transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 11, pp. 17366–17381, Nov. 2024.
- [28] J. Li, J. Li, D. Xie, and Z. Cai, "Secure auditing and deduplicating data in cloud," *IEEE Trans. Comput.*, vol. 65, no. 8, pp. 2386–2396, Aug. 2016.
- [29] Q. Xie, C. Zhang, and X. Jia, "Security-aware and efficient data deduplication for edge-assisted cloud storage systems," *IEEE Trans. Services Comput.*, vol. 16, no. 3, pp. 2191–2202, May/Jun. 2023.
- [30] T. Jiang et al., "FuzzyDedup: Secure fuzzy deduplication for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 2466–2483, May/Jun. 2023.
- [31] Y. Zhang, C. Xu, N. Cheng, and X. Shen, "Secure password-protected encryption key for Deduplicated cloud storage systems," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 4, pp. 2789–2806, Jul./Aug. 2022.
- [32] S. Halevi, D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Proofs of ownership in remote storage systems," in *Proc. 18th ACM Conf. Comput. Commun. Secur. (CCS)*, 2011, pp. 491–500.
- [33] J. Yuan and S. Yu, "Secure and constant cost public cloud storage auditing with deduplication," in *Proc. IEEE Conf. Commun. Netw. Security (CNS)*, 2013, pp. 145–153.
- [34] Q. Zhang, D. Sui, J. Cui, C. Gu, and H. Zhong, "Efficient integrity auditing mechanism with secure deduplication for blockchain storage," *IEEE Trans. Comput.*, vol. 72, no. 8, pp. 2365–2376, Aug. 2023.

- [35] Y. Miao, K. Gai, L. Zhu, K.-K. R. Choo, and J. Vaidya, "Blockchain-based shared data integrity auditing and deduplication," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 3688–3703, Jul./Aug. 2024.
- [36] X. Zheng, W. Shen, Y. Su, and Y. Gao, "DIADD: Secure deduplication and efficient data integrity auditing with data dynamics for cloud storage," *IEEE Trans. Netw. Service Manag.*, vol. 22, no. 1, pp. 299–316, Feb. 2025.
- [37] F. Falzon, K. Elkhyaoui, Y. Manevich, and A. D. Caro, "Short privacy-preserving proofs of liabilities," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2023, pp. 1805–1819.
- [38] A. K. Lenstra and B. Wesolowski, "Trustworthy public randomness with sloth, unicorn, and trx," *Int. J. Appl. Cryptogr.*, vol. 3, no. 4, pp. 330–343, 2017.



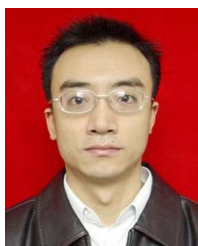
Bingyun Liu received the B.E. degree in electronic and computer engineering from Southwest Petroleum University, Chengdu, China, in 2023. He is currently pursuing the graduation in cyberspace security with the School of Computer Science and Software Engineering.

His research interests include cryptography, data auditing, and encrypted deduplication.



Xiaojun Zhang received the Ph.D. degree in information security from the University of Electronic Science Technology of China, Chengdu, China, in 2015.

He is currently a Professor with Southwest Petroleum University, Chengdu. He was a research scholar from 2018 to 2019 with Nanyang Technological University, Singapore. His research interests include applied cryptography, big data, and cloud computing security.



Xingchun Yang received the Ph.D. degree from the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2017.

He is currently a Professor with the Department of Computer Science and Technology, Sichuan Police College, Luzhou, China. His research interests include information security and data mining.



Yuan Zhang (Member, IEEE) received the B.Sc. and Ph.D. degrees from the University of Electronic Science Technology of China (UESTC), Chengdu, China, in 2013 and 2019, respectively.

He was a visiting Ph.D. student from 2017 to 2019 with BCCR Lab, Department of ECE, University of Waterloo, Waterloo, ON, Canada. He is currently an Assistant Professor with School of Computer Science and Engineering, UESTC. His research interests are applied cryptography and data security.



Jingting Xue received the Ph.D. degree in cyber security from the University of Electronic Science Technology of China, Chengdu, China, in 2020.

She had a one-year study and research with Nanyang Technological University, Singapore, in 2019. She is currently a Lecturer with the School of Computer Science and Software Engineering, Southwest Petroleum University, Chengdu. Her research interests are applied cryptography, cloud storage, and blockchain technology.



Rang Zhou received the B.S., M.S., and Ph.D. degrees from the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2009, 2013, and 2020, respectively.

He is currently an Associate Professor with the College of Computer Science and Cyber Security, Chengdu University of Technology, Chengdu. His research interests include cryptography, access control security, and data security.